

Optimierung und Erweiterung eines webbasierten Frameworks für interaktive 3D-Exploded-View-Animationen

Maximilian Elias Kellner

Modul: Bachelorarbeit

Erstprüfer & Betreuer: Prof. Christian Noss

Zweitprüfer: Prof. Dr. Hoai Viet Nguyen

21. Februar 2026

Zusammenfassung

Interaktive 3D-Exploded-View-Animationen sind ein wesentlicher Bestandteil moderner technischer Dokumentationen und Produktdarstellungen im Web. Trotz leistungsfähiger Bibliotheken wie Three.js bleibt die Erstellung solcher Visualisierungen für Entwickler oft komplex, zeitaufwendig und fehleranfällig. Diese Bachelorarbeit befasst sich mit der Optimierung und funktionalen Erweiterung meines Frameworks „Explo.js“, um die Einstiegshürde für Anwender zu senken und den Erstellungsprozess effizienter zu gestalten.

Dafür werden etablierte Interaktionsmuster aus professioneller Kreativsoftware und bestehende UI-Prinzipien untersucht und implementiert. Darauf aufbauend wurde ein interaktiver WYSIWYG-Editor entwickelt, der die bisherige rein codebasierte JSON-Konfiguration durch eine visuelle Benutzeroberfläche ersetzt.

Inhaltsverzeichnis

1	Einleitung	4
2	Theoretischer und praktischer Hintergrund	6
2.1	Exploded Views und interaktive 3D-Visualisierung	6
2.2	Workflows für 3D-Visualisierungen	6
2.3	Usability und Tool-Design	7
2.3.1	Grundbegriffe	7
2.3.2	Bedeutung von visuellen Editoren	7
2.4	Etablierte 3D-Software als Referenz	8
2.4.1	Gizmos	8
2.4.2	Transform Controls	9
2.4.3	Timeline	10
2.4.4	Keyframes	10
2.4.5	Vorschaumodi	10
2.4.6	Komprimierte Listen	12
3	Methodischer Rahmen	13
3.1	Allgemeines Vorgehensmodell	13
3.2	UI-Designprozess	13
3.2.1	Analyse und Anforderungsdefinition	13
3.2.2	Low-Fidelity-Wireframe	13
3.2.3	High-Fidelity UI	14
3.2.4	Implementierung	14
4	Analyse des bestehenden Frameworks	15
4.1	Ausgangszustand	15
4.1.1	Funktionsumfang des Frameworks	15
4.1.2	Stärken	15
4.1.3	Bestehender Workflow	19
4.2	Identifizierte Probleme und Defizite	22
4.3	Auswirkungen auf Usability	23
5	Konzeption der Optimierungen und Erweiterungen	24
5.1	Designziele	24
5.2	Editor-Konzept	24
5.2.1	Rolle des Editors	24
5.2.2	Grundfunktionen des Editors	24
5.2.3	Abgrenzung	26
5.3	Erweiterte Animations- und Konfigurationsmöglichkeiten	26
5.3.1	Neue Optionen	26
5.3.2	Vereinfachungen	26
6	Umsetzung	28
6.1	Technische Architektur	28
6.1.1	Integration des Editors in das Paket	28
6.1.2	Datenfluss und Events	28
6.1.3	Deployment- und Build-Prozess	29
6.2	Implementierung des Editors	29

6.2.1	Konfigurationslogik	29
6.2.2	Übertragene UI-Konzepte aus Kreativsoftware	29
6.3	Erweiterte Animationslogik	34
6.3.1	Intervalle	34
6.3.2	Easing	35
6.3.3	Rotation	35
6.3.4	Kamera	35
6.3.5	Lichter	36
6.4	Beispiel-Workflows	36
6.4.1	Allgemeiner Workflow	36
6.4.2	Onboarding	38
7	Ergebnisse und Diskussion	40
7.1	Validierung der Effizienzsteigerung	40
7.1.1	Versuchsaufbau	40
7.1.2	Methodik	40
7.1.3	Daten und Analyse	41
7.2	Ergebnisse	41
7.3	Bewertung	41
7.4	Limitationen	42
8	Fazit und Ausblick	43
8.1	Zusammenfassende Beantwortung der Forschungsfragen	43
8.2	Zusammenfassung	43
8.3	Ausblick	44

1 Einleitung

Kontext

Leistungsfähige Prozessoren sowie die kontinuierliche Weiterentwicklung moderner Webtechnologien ermöglichen die Darstellung interaktiver Elemente und komplexer 3D-Szenen im Browser. Frameworks wie Three.js bilden dabei die Grundlage für viele Projekte und ermöglichen außergewöhnliche Inhalte, welche die Nutzerwahrnehmung deutlich verbessern können (Kellner, 2025d).

Ein konkreter Anwendungsfall sind Exploded View Animationen. Diese ermöglichen es, komplexe Objekte zu zerlegen und deren Aufbau und funktionale Zusammenhänge zu visualisieren. Dadurch eignet sich diese Form der Visualisierung besonders für technische Dokumentationen, Anleitungen und Produktdarstellungen.

Problemstellung

Trotz dieser technischen Möglichkeiten zeigt sich bei der praktischen Anwendung, dass die Entwicklung einer maßgeschneiderten Lösung oft sehr komplex und zeitaufwändig sein kann. Three.js bietet einen großen Funktionsumfang, jedoch wenig Spezialisierung. Dadurch müssen Nutzer eigene Konzepte erarbeiten und entwickeln. Dabei wird der überwiegende Teil im Code spezifiziert und es gibt wenig visuelles Feedback und keinen klaren Workflow.

Um dieses Problem zu lösen, wurde im Praxisprojekt **Konzeption und Entwicklung eines interaktiven (Visualisierungs-)Systems für Exploded Views im Web.** (Kellner, 2025d) ein Paket entwickelt, welches das Erstellen von Exploded Views stark erleichtert und standardisiert. Das Paket übernimmt die Konfiguration und Integration der 3D-Umgebung und ermöglicht eine vereinfachte und strukturierte Erstellung der Animation. Dadurch muss der Nutzer lediglich das gewünschte 3D-Modell angeben und die Animation über Vektoren und Distanzen in einer JSON-Datei konfigurieren. Diese Abstraktion vereinfacht das Erstellen von Exploded Views maßgeblich und ermöglicht es, 3D-Modelle und Animationen auszutauschen.

Jedoch sind auch bei diesem Paket noch große Verbesserungen möglich. Besonders das Erstellen der ersten Animation mit dem Paket ist mit einer steilen Lernkurve verbunden, da das entwickelte Konzept mit Vektoren, Distanzen und Geschwindigkeiten verstanden werden muss und keine visuelle Vorschau existiert. Zudem sind Nutzer durch den Funktionsumfang des Pakets begrenzt. Es werden keine komplexen Skalierungen, Rotationen, Kamerafahrten oder frei wählbare Intervalle und Keyframes ermöglicht.

Das Ziel dieser Arbeit ist es daher, die Lernkurve und Fehleranfälligkeit der Konfiguration mit einem interaktiven Editor zu reduzieren und so die Nutzbarkeit des Pakets zu verbessern. Ein Editor mit integrierter Vorschau vereinfacht und beschleunigt die Konfiguration. Zudem soll der Funktionsumfang ausgeweitet werden, um komplexere und visuell ansprechendere Animationen zu ermöglichen. Um diese Verbesserungen strukturiert zu erzielen, werden folgende Forschungsfragen untersucht und beantwortet.

- *Welche zentralen Probleme und Defizite von `Explo.js` beeinflussen den Workflow und die Nutzbarkeit, und wie lassen sich diese durch gezielte konzeptionelle und technische Anpassungen verbessern?*

- *Welche Visualisierungs- und Interaktionsmuster aus etablierter 3D-Software lassen sich sinnvoll auf einen webbasierten Exploded-View-Editor übertragen, um die Konfiguration, Steuerung und Vorschau von Animationen zu verbessern?*

2 Theoretischer und praktischer Hintergrund

In diesem Kapitel werden kontextrelevante Workflows und Konzepte wissenschaftlich analysiert und bewertet.

2.1 Exploded Views und interaktive 3D-Visualisierung

Bei Exploded Views handelt es sich um eine Darstellungsform, bei der ein Objekt in seine relevanten Komponenten zerlegt wird. Die Komponenten werden dann auf vordefinierten Achsen aufgereiht. Dadurch wird es möglich, die räumliche Anordnung und das funktionale Zusammenspiel der Komponenten zu visualisieren und somit den Aufbau, die Beziehungen und die Funktionen der Komponenten innerhalb eines komplexen Objekts nachvollziehbar darzustellen (Kellner, 2025d, S. 5).

Besonders im Web können durch Animationen und Interaktionsmöglichkeiten die Anwendungsfälle und Funktionen maßgeblich erweitert werden. Zudem scheinen Nutzer Webseiten mit interaktiven Elementen als hochwertiger wahrzunehmen (Summerlin und Powell, 2022). Diese positive Wahrnehmung und auch die kognitive Forderung des Nutzers können sich positiv auf das Kaufverhalten auswirken (Jiang et al., 2010). Interaktive Elemente auf Webseiten bieten demnach mehr als ein grafisch ansprechendes Alleinstellungsmerkmal.

2.2 Workflows für 3D-Visualisierungen

1. **Erstellen des 3D-Modells:** Um ein 3D-Modell zu erstellen wird als Basis eine Anwendung, wie beispielsweise Blender oder Maya3D verwendet. Diese ermöglichen das Erstellen, Importieren, Texturieren, Animieren, Rendern und Exportieren der Modelle. Diese Programme bieten große kreative Freiheiten und ermöglichen feine Anpassungen an Modell und Szene.
2. **Exportieren:** Für den Export gibt es verschiedene Möglichkeiten. 3D-Szenen können gerendert werden, dadurch entstehen Bilder im Rasterformat. Diese können mit geringem Aufwand komprimiert oder bearbeitet und wiederverwendet werden. Dadurch wird das 3D-Modell jedoch zu einem statischen Bild mit fester Auflösung und fehlender Interaktivität. Bildfolgen können zwar interaktiv implementiert werden, bei Änderungen muss jedoch meist die gesamte Bildfolge neu gerendert werden.
Eine Alternative ist der Export der gesamten Modelle als 3D-Datei. GLB-Dateien sind vor allem im Web-Kontext weit verbreitet und haben sich als Standard etabliert. Sie können 3D-Modelle mit Texturen und anderen Daten kapseln. Während die Geometrie und die Materialien meist reibungslos übertragen werden, können bei der Darstellung dieser Probleme auftreten. Abweichende Shader, Koordinatensysteme und andere Konfigurationen können für einen hohen Iterationsaufwand sorgen, da diese Probleme erst bei der Zielplattform sichtbar werden. Um diese Probleme zu minimieren, sollten die Lichtquellen in der 3D-Umgebung der Zielplattform konfiguriert werden. Alternativ können Lichtquellen über **Baking** in **lightmaps** umgewandelt werden. Dieser Prozess erfordert mehr Konfiguration und Rechenleistung beim Erstellen des 3D-Modells, kann die Bildwiederholrate jedoch besonders bei komplexen Szenen signifikant verbessern (Loser, 2015). **Lightmaps** erhöhen jedoch den Speicherverbrauch

und erzeugen besonders bei dynamischen Objekten neue Probleme, welche mit hybriden Ansätzen umgangen werden müssen (Ozturk und Akyuz, 2010).

3. **Zielpattform:** Die Zielpattform stellt in diesem Fall Three.js beziehungsweise das entwickelte Paket `Explo.js` dar. Hier müssen die potenziellen Medienbrüche, Belichtung, Rotation, Skalierung und Position überprüft werden. Zudem wird hier die Szene und die Animation konfiguriert. Diese Konfiguration erfordert momentan viele Iterationen, obwohl die Namenskonvention bereits durch eine austauschbare JSON-Konfiguration ersetzt wurde (Kellner, 2025c), müssen Nutzer Objekte anhand des Namens identifizieren und diesen Objekten ohne eine visuelle Vorschau Vektoren und Distanzen zuordnen.

2.3 Usability und Tool-Design

2.3.1 Grundbegriffe

Die Usability ist ein wichtiger Bereich im Softwaredesign. Die ISO 9241-11 misst diese anhand der Effektivität, Effizienz und der Zufriedenstellung, welche die Nutzer erfahren. Es soll möglich sein, Ziele vollständig und genau zu erreichen und dabei den Aufwand in einem akzeptablen Verhältnis zu halten. Zudem sollte der Nutzer nicht negativ beeinträchtigt werden und allgemein eine positive Einstellung gegenüber dem Programm beibehalten (International Organization for Standardization, 2018).

Ergänzend ist auch eine niedrige Einstiegshürde wichtig. Der Einstieg in das Programm muss einfach und verständlich gestaltet werden, damit Nutzer nicht abgeschreckt werden. Nutzer sollten schnell einen Überblick über die Grundfunktionen bekommen und diese anwenden können, sodass sie das Programm schnell produktiv nutzen können. Eine hohe Einstiegshürde und ein langer Onboarding Prozess kann die Nutzerzahlen nachweislich reduzieren (ABBYY Europe GmbH, 2022). Zudem sollten weitere Ressourcen zur Dokumentation, Erklärung und Beispiele für komplexere Anwendungsfälle bereitgestellt werden, sodass der Nutzer auch nach dem Einstieg schnell weitere Funktionen erlernen und anwenden kann. So kann gewährleistet werden, dass die Lernkurve nicht zu früh abflacht und Nutzer das volle Potenzial des Programms ausschöpfen können (Nielsen, 2024).

2.3.2 Bedeutung von visuellen Editoren

Besonders bei Editoren muss eine gute Usability gewährleistet sein. Sie können jedoch auch ein wichtiger Bestandteil des Workflows sein, der die Usability maßgeblich verbessert. Um einen guten Editor zu entwickeln, sind diverse Best Practices und Konzepte zu beachten, da die Qualität des Editors direkt mit der Ergebnisqualität zusammenhängt. Zu diesen Konzepten zählt beispielsweise **What You See Is What You Get** (WYSIWYG). Dieses Akronym beschreibt die akkurate Vorschau in Echtzeit. Das Ergebnis der aktuellen Eingabe sollte schnell und akkurat ausgegeben werden, sodass Nutzer verifizieren können, ob das gewünschte Ergebnis erreicht wurde (Myers, 1998, Shneiderman, 1983). Besonders in digitalen 3D-Umgebungen ist dieser Ansatz der direkten Manipulation

und Vorschau sehr wichtig, da dreidimensionale Inhalte auf einem zweidimensionalen Medium angezeigt werden. Um diese Einschränkung zu überwinden, werden hier erprobte UI-Patterns verwendet. Diese sollten konsistent angewendet werden und erleichtern so den Einstieg in das Programm, da Nutzer diese Muster erkennen (Nielsen, 2024). Nutzer sollten die meiste Zeit mit dem Erstellen der Inhalte verbringen und wenig über die Benutzung der Software nachdenken müssen (Krug, 2014).

2.4 Etablierte 3D-Software als Referenz

Ein guter Einstieg in das UI-Design ist die Analyse von bestehenden Anwendungen und Webseiten. Dabei fallen Muster auf, welche immer wieder verwendet werden. Einige dieser Muster werden in diesem Abschnitt referenziert und sollten zur Verringerung der Nutzerbelastung und schnellen Adaption implementiert werden (Nielsen, 2024, Krug, 2014).

Für diesen Abschnitt wurden diverse Kreativprogramme zur Erstellung von Videos, Grafiken, Animationen und 3D-Modellen analysiert und die für den Editor und den 3D-Kontext relevantesten Muster herausgestellt.

Folgende Software wurde hierbei untersucht:

- **3D-Modellierung & Engine:** Blender, Unity 3D, AutoCAD, Microsoft 3D-Viewer
- **Videoschnitt & Animation:** DaVinci Resolve, Microsoft Clipchamp
- **Webbasierte Tools:** Three.js, shapeshifter.design

2.4.1 Gizmos

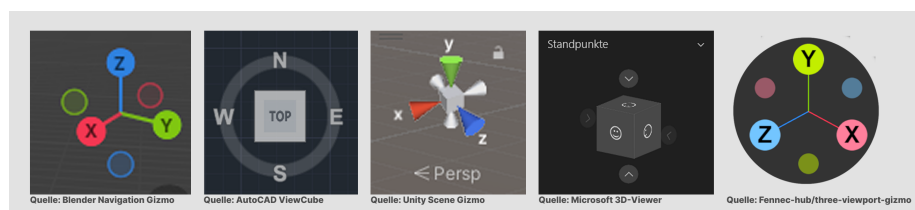


Abbildung 1: Verschiedene Gizmo und View-Cube Elemente (Quellen unter den Bildern).

3D-Gizmo-Elemente werden in diversen Programmen mit 3D-Kontext verwendet (vgl. Abb. 1). Dieses Design-Muster wird verwendet, um die Orientierung der Nutzeransicht im Verhältnis zum globalen Koordinatensystem der Szene zu visualisieren. Das Navigations-Gizmo ist meist permanent in der oberen rechten Bildschirmcke sichtbar. Das zentrale Element ist meist ein Koordinatensystem mit hervorgehobenen Primärachsen (positive X-, Y-, Z-Achse), alternativ werden beschriftete Würfel, sogenannte View-Cubes, verwendet. Neben der Orientierung in der 3D-Szene ermöglichen diese Objekte meist die freie Rotation der Nutzeransicht. Bei der Auswahl einer Achse wird die Szene typischerweise

orthografisch projiziert dargestellt. Dabei wird die Szene mit parallelen Projektionsstrahlen senkrecht zur Projektionsebene abgebildet. Dadurch entsteht eine maßhaltige, verzerrungsfreie 2D-Darstellung (vgl. International Organization for Standardization, n. d.). Diese Darstellungsform erleichtert den Vergleich von Größenverhältnissen.

2.4.2 Transform Controls

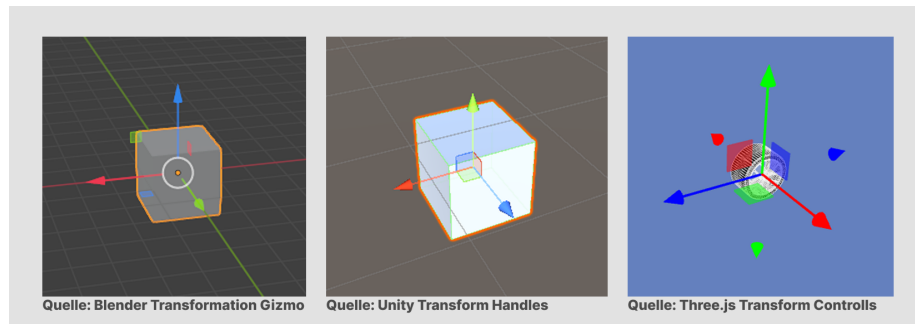


Abbildung 2: Verschiedene Implementierungen von Transform Controls (Quellen unter den Bildern).

Transform Controls erfüllen einen ähnlichen Funktionsumfang wie Szenen-Gizmos (vgl. 2.4.1). Diese Elemente werden jedoch zur Manipulation der Objekte in der Szene verwendet. Die Grundfunktion besteht im Verschieben von Objekten (vgl. Abb. 3.1). Wenn ein 3D-Objekt ausgewählt wird, werden die Transform Controls eingeblendet. Dann kann das Objekt anhand der angezeigten Achsen verschoben werden. Alternativ können die Quadrate zum Verschieben auf einer Ebene zwischen zwei Achsen verwendet werden. Das Zentrum der Transform Controls ermöglicht freie Verschiebungen im Verhältnis zur Nutzerkamera. Andere Optionen umfassen das Rotieren, Skalieren oder eine Kombination dieser (vgl. Abb. 3.2-4). Die Grundlagen dieses UI-Musters sind also die Visualisierung von Achsen und Ebenen sowie die Manipulation von Werten eines spezifischen Objekts.

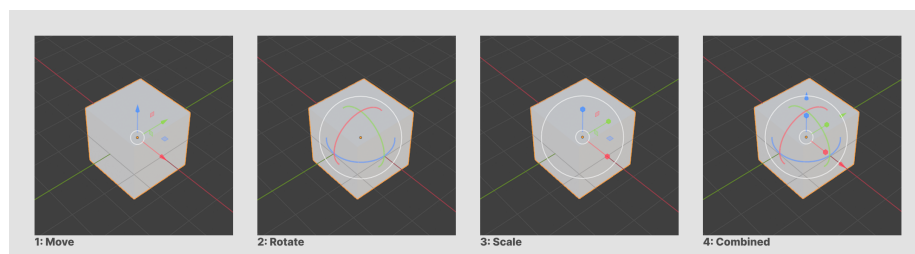


Abbildung 3: Verschiedene Optionen von Transform Controls in Blender (Quellen: Blender Screenshots).

2.4.3 Timeline

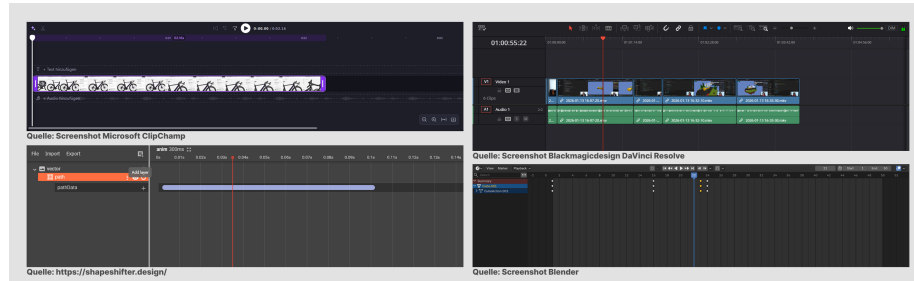


Abbildung 4: Timelineelemente aus Verschiedenen Programmen und Diensten (Quellen unter den Bildern).

Timelineelemente werden in vielen Kreativprogrammen verwendet, um eine kompakte Übersicht zu schaffen, welche die Steuerung und Visualisierung der Animation oder Zeit ermöglicht. Dabei verfügt die Timeline meistens über eine Nadel, auch Scrubber genannt, welche die Auswahl eines bestimmten Zeitpunkts erlaubt und die volle Höhe der Timeline einnimmt, um die aktuell relevanten Objekte zu markieren. Besonders bei parallel ablaufenden Inhalten, wie Keyframes oder Ton- und Bildspuren, wird so ein guter Überblick ermöglicht. Aufgrund der Kapselung der Steuerung von Zeit und Inhalten können hier semantisch passende Funktionen wie Play/Pause Funktionen, Einstellungen zur Dauer sowie Import- und Export-Möglichkeiten integriert werden.

2.4.4 Keyframes

Keyframes sind ein fester Bestandteil von Kreativprogrammen. Sie ermöglichen die Speicherung von Zuständen zu bestimmten Zeitpunkten. Dabei kann beispielsweise die Position zu einem Zeitpunkt X und zu einem Zeitpunkt Y angegeben werden. Das Programm interpoliert dann eine Bewegung zwischen diesen Positionen, sodass das ausgewählte Objekt zum Zeitpunkt X an Position eins und zum Zeitpunkt Y an Position zwei ist. Um die Korrelation zwischen Objekteigenschaften und Zeitpunkten in grafischen Benutzeroberflächen zu ermöglichen, werden Keyframes meist als Punkte oder Intervalle in Timeline-Elementen integriert (vgl. Abb. 4.3-4). Zudem werden Keyframes oft mit Bewegungskurven kombiniert, um dem Nutzer trotz der Abstraktion der Bewegung Kontrolle über Beschleunigung und Entschleunigung sowie andere visuelle Effekte zu ermöglichen.

2.4.5 Vorschau Modi

Vorschau Modi umfassen verschiedene Filteroptionen. Dabei kann meist zwischen Wireframe, Solid, Material Preview und Rendered gewählt werden. Diese Funktionen werden meist durch Optionen zur Transparenz ergänzt.

Die Wireframe-Ansicht blendet die Flächen aus und stellt nur die bestehenden Punkte und Kanten des 3D-Modells dar (vgl. Abb. 5.1), wodurch ein Abgleich von Symmetrien und Positionen möglich wird. Dieser Modus ist besonders für

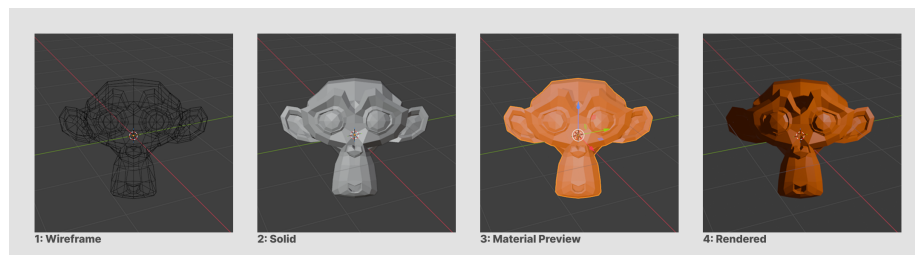


Abbildung 5: Verschiedene Vorschaumodi (Quellen: Blender Screenshots).

den Vergleich von Details wichtig.

Die Solid-Ansicht stellt das Modell mit einem ausgewählten Standardmaterial dar (vgl. Abb. 5.2). Dieser Modus verbraucht durch den Verzicht auf weitere Materialien und Texturen vergleichsweise wenig Leistung und ermöglicht eine gute Übersicht über Volumen und Mesh-Topologie. Einige Programme bieten Optionen, um spiegelnde Materialien oder wiederkehrende Muster als Standardmaterial auszuwählen. Diese Materialien umfassen auch Konfigurierte Lichter oder Shader (vgl. Abb. 6). Dadurch können Shading- oder Textur-Fehler bereits bei der Modellierung verhindert werden.

Die Materialvorschau ermöglicht es eine grobe Übersicht über die Farbgebung und Materialien der Szene zu bekommen (vgl. Abb. 5.3). Dieser Modus visualisiert die wichtigsten Eigenschaften der jeweiligen Texturen, ohne die Beleuchtung und komplexere Materialien wie beispielsweise Glas zu berücksichtigen.

Die letzte Ansicht ermöglichte eine hochwertige und umfangreiche Vorschau der gesamten 3D-Szene (vgl. Abb. 5.4). Das Programm nutzt die ausgewählte Renderengine und erstellt eine Vorschau unter Berücksichtigung aller Materialien und Lichtquellen. Diese Darstellungsform beansprucht am meisten Ressourcen, wird jedoch für finale Abstimmungen der Materialien und der Komposition benötigt. Diese Darstellung kommt dem gerenderten Ergebnis am nächsten. Die Ausgabe ist jedoch von der ausgewählten Renderengine abhängig. Wie bereits in Kapitel 2.2 beschrieben, wird das 3D-Modell in anderen 3D-Programmen und Szenen möglicherweise anders dargestellt.

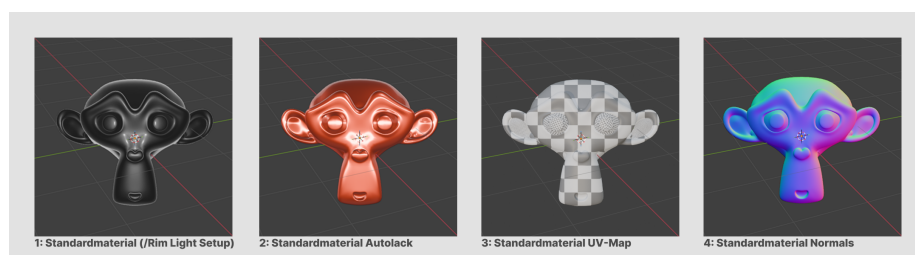


Abbildung 6: Auswahl weiterer Materialien mit Lichtquellen (Quellen: Blender Screenshots).

2.4.6 Komprimierte Listen

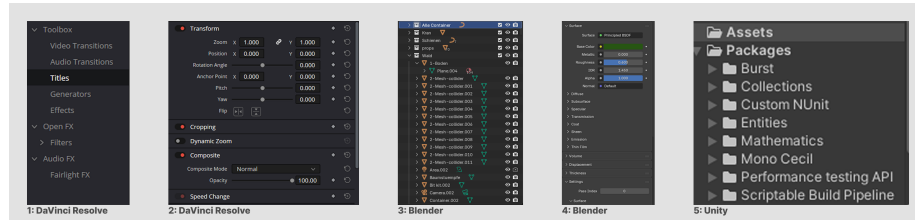


Abbildung 7: Komprimierte Auflistungen (Quellen: Unter den Bildern).

In Kreativprogrammen wird eine hohe Informationsdichte mit diversen Parametern und Zahlen dargestellt. Um diese Datenmengen effizient und übersichtlich darzustellen, werden Listen mit Überschriften und ausklappbaren Bereichen angewendet (vgl. Abb. 7). Diese Listen fassen mehrere Inhalte in Ordnern oder Kategorien zusammen und ermöglichen es, ganze semantische Abschnitte auszublenken oder zu deaktivieren.

3 Methodischer Rahmen

In diesem Kapitel werden das methodische Vorgehen in der Entwicklung und im Gestaltungsprozess beschrieben.

3.1 Allgemeines Vorgehensmodell

Die Ergebnisse dieser Projektarbeit wurden mit einem iterativen, prototypischen Ansatz entwickelt. Das Ziel dieser Projektarbeit ist primär die Verbesserung des entwickelten Pakets zur Erstellung von Animationen im Web. Dafür wurden Problemfelder, Anforderungen und Ziele ermittelt.

Um die Komplexität der Entwicklung zu beherrschen, wurden die neuen Komponenten ebenfalls modular aufgebaut. Kritische Komponenten, deren Umsetzbarkeit unklar war oder deren Funktion kritisch für den Projekterfolg war, wurden gesondert in Proof of Concepts (POCs) entwickelt. Dabei wurden vorher systematisch das Problem, der Umfang, die benötigten Ressourcen, das Ziel, klare Erfolgskriterien sowie mögliche fallbacks für Misserfolge festgelegt (vgl. Kellner, 2026b). Nach iterativer Ausarbeitung wurde der POC, sofern alle relevanten Ziele erreicht werden konnten, in den Prototyp integriert und dort weiterentwickelt und verfeinert. Dadurch kann sichergestellt werden, dass die mit den Personas erarbeiteten Userflows und Anforderungen abgedeckt und erfüllt werden.

3.2 UI-Designprozess

Der Designprozess der UI-Elemente wurde in verschiedene Phasen aufgeteilt. Dabei wurde der Prozess des User-Centered Design verfolgt. Das Ziel bei diesem Prozess ist es, den Nutzer und den Nutzungskontext zu verstehen, darauf basierend Anforderungen zu entwickeln und diese im Design zu erfüllen. Die Bedürfnisse potenzieller Nutzer beeinflussen demnach direkt die Designentscheidungen.

3.2.1 Analyse und Anforderungsdefinition

Als Grundlage der Analyse wurden Gespräche mit potenziellen Nutzern geführt. Dabei wurden verschiedene vollständig konfigurierte Beispiele präsentiert, um einen Überblick über verschiedene Ergebnisse und Anwendungsfälle zu erlangen. Zudem wurde die Konfiguration und der Installationsprozess erklärt. Ein Proband hat das Projekt beispielhaft implementiert und konfiguriert. Bei diesen Gesprächen und Tests wurden Designentscheidungen wie Abläufe, die Nutzererfahrung und Architekturentscheidungen diskutiert. Dadurch konnten Vermutungen zu bestehenden Problemen bestätigt und neue Probleme identifiziert werden. Zudem wurden verschiedene Personas in relevanten Zielgruppen konzipiert (vgl. Kellner, 2026a). Ebenfalls wurden bestehende Workflows (2.2), Studien und Best Practices (2.3) und etablierte Softwareprodukte (2.4) analysiert. Dadurch konnten Designziele, wichtige UI-Muster und Design-Attribute erarbeitet werden.

3.2.2 Low-Fidelity-Wireframe

Die gewonnenen Erkenntnisse wurden verwendet, um einen Low-Fidelity-Wireframe für die verschiedenen Ansichten der Anwendung zu erstellen. Dieser fokussiert

sich jeweils auf das Layout der UI-Elemente und die benötigten Funktionen.

3.2.3 High-Fidelity UI

In diesem Schritt wurden die Erkenntnisse aus dem Low-Fidelity-Wireframe, die Designziele, wichtige UI-Muster und Design-Attribute gesammelt und iterativ zu einem High-Fidelity-Wireframe kombiniert. Es wurden eine Designsprache, eine Farbpalette und weitere Details ausgearbeitet, um eine klare Designvorlage für die Entwicklung zu erhalten.

3.2.4 Implementierung

Bei der Implementierung des Designs wurden finale Anpassungen, Microinteraktionen und Funktionen ergänzt, um das Nutzerfeedback und den Systemstatus besser zu Veranschaulichen (vgl. Kendrick, 2018).

4 Analyse des bestehenden Frameworks

In diesem Kapitel wird der Ausgangszustand des bestehenden Frameworks systematisch untersucht, um Probleme und Defizite zu identifizieren und deren negative Auswirkungen auf die Usability zu verringern oder bessere Alternativen zu implementieren.

4.1 Ausgangszustand

4.1.1 Funktionsumfang des Frameworks

Das im Praxisprojekt entwickelte Framework (Kellner, 2025d) verfügt bereits über einen großen Funktionsumfang, welcher das Erstellen, Integrieren und Konfigurieren von Exploded-View-Animationen im Web vergleichsweise einfach ermöglicht. Ausgehend von einem 3D-Modell kann eine Animation über Vektoren und Ebenen erstellt werden, diese kann vom Nutzer über verschiedene Trigger präzise gesteuert werden. Zudem können verschiedene Arten von Beschriftungselementen für die einzelnen Komponenten hinzugefügt werden, welche in Kombination mit einer Auswahl an Visualisierungen zur Hervorhebung des Erkunden und Verstehen des 3D-Modells ermöglichen und fördern. Ebenfalls bietet das Framework eine Debug-UI, welche Entwicklern das schnelle Iterieren von diversen Szenenparametern, Interaktionen, Kameraeinstellungen, Farbgebungen und anderen Konfigurationen ermöglicht (vgl. Kellner, 2025d 5.2).

4.1.2 Stärken

Die Stärken belaufen sich besonders auf die vergleichsweise einfache Integration, das konzipierte Animationssystem, die umfangreichen Interaktions- und Konfigurationsmöglichkeiten und die ermöglichten Anwendungsfälle.

Integration: Nach der Installation und Konfiguration kann das Paket vergleichsweise einfach integriert werden (vgl. Listing 1). Die benötigten Optionen und Dateien können in einem Objekt gebündelt und an den Exploded Viewer übergeben werden. Dadurch können verschiedene Konfigurationen parallel geführt und gut verwaltet und ausgetauscht werden.

Listing 1: Konfigurationsbeispiel des Exploded Viewers

```
1 // Kopfhoerer-Animation-Konfiguration
2 const kopfhoerer = {
3
4   // Szenenkonfiguration
5   sceneConfigPath: '/kopfhoerer/scene-config.json',
6
7   // .glb Modell
8   modelPath: '/kopfhoerer/kopfhoerer-modell.glb',
9
10  // Explosions-Konfiguration
11  explosionConfigPath: '/kopfhoerer/exp-config.json',
12
13  // Beschriftungen
14  cardDataPath: '/kopfhoerer/Kopfhoerer-cards.json',
15 }
```

```

16 //Hier koennen optional weitere Parameter konfiguriert
    werden.
17     showDebugUI: true,
18     showStats: true,
19 }
20
21 const expViewer = new ExplodedViewer(container, expOptions);

```

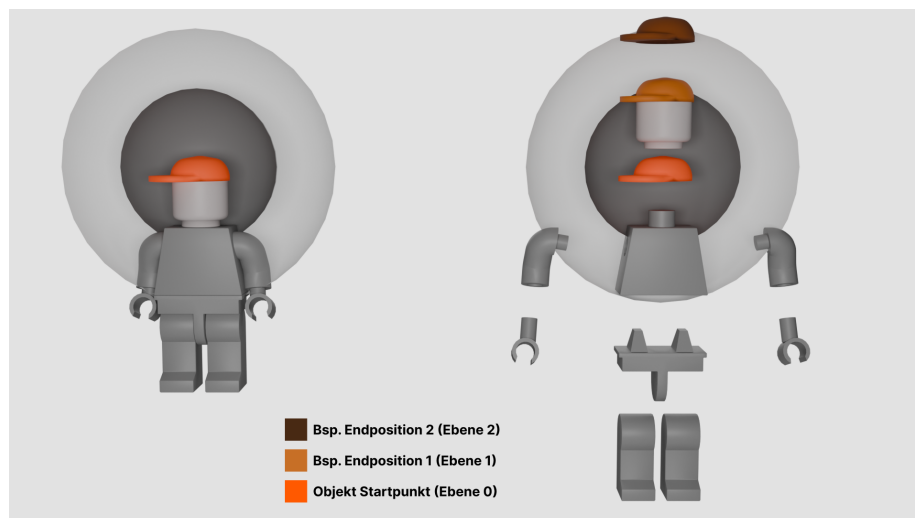


Abbildung 8: Visualisierung möglicher Endpunkte mit Vektor `expDirection`: $(x:0|y:1|z:0)$. Die Kreise visualisieren Ebene 1 (Dunkel) und Ebene 2 (Hell).

Animationssystem: Das konzipierte Animationssystem ermöglicht eine präzise Konfiguration. Mit einer Kombination aus einem Vektor `expDirection` und einer Ebene `level` kann die Grundlage der Animation gesetzt werden (vgl. Abb.8). `expDirection` gibt dabei die Richtung der Animation an, dieser Vektor kann frei gewählt werden und wird normalisiert. Um die Distanz des Objekts zwischen Start- und Endpunkt zu konfigurieren, wird `level` verwendet. Dabei handelt es sich um eine frei wählbare (Dezimal-)Zahl, welche mit `expDirection` verrechnet wird. Diese Kombination ermöglicht es, Ebenen um ein Objekt zu bilden und Objekte auf einer Achse auf die gleiche Endposition zu bringen (vgl. Abb.9). Zudem sind symmetrische Animationen intuitiver und einfacher zu konfigurieren. Die Entkopplung von Koordinaten vereinfacht diesen Prozess weiter. Weitere Parameter sind `sequence` und `speedMultiplier`. Diese Optionen ermöglichen visuell ansprechendere Animationen und erlauben Animationen für mehrschichtige Objekte, ohne Überlappungen und Kollisionsfehler zwischen den 3D-Objekten. `speedMultiplier` beeinflusst die Geschwindigkeit, in der ein Objekt die Endposition erreicht. Dadurch kann dieses Objekt die Animation schneller abschließen als parallel animierte Objekte. `sequence` ermöglicht, dass Objekte in verschiedenen Intervallen animiert werden. Alle Objekte werden nur in ihrem ausgewählten

Intervall animiert, Objekte ohne `sequence` werden im $n+1$ ten Intervall gleichzeitig animiert (vgl. Abb.10)

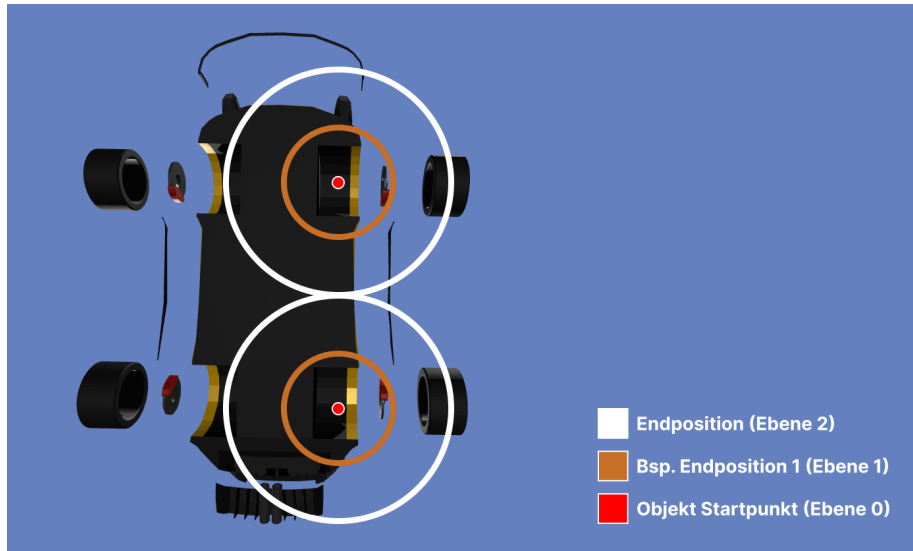


Abbildung 9: Animation mit zwei Rädern, welche die gleiche Ebene `level` und den gleichen Vektor `expDirection` haben. Die Kreise visualisieren Ebene 1 und Ebene 2, Die Endposition wird erreicht, da Ebene 2 ausgewählt ist und der Vektor in X Richtung zeigt. (Quellen: Screenshot aus der Anwendung).

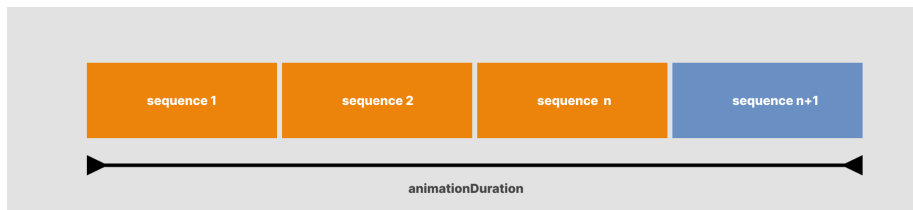


Abbildung 10: Aufteilung der Sequenzen auf die Animationsdauer. Alle Objekte ohne `sequence` werden im $n+1$ ten Intervall gesammelt und gleichzeitig animiert

Konfiguration: Die umfangreichen Konfigurationsmöglichkeiten können größtenteils in Echtzeit in der Anwendung erprobt werden, nachdem `showDebugUI: true` in den Optionen (vgl. Listing 1) oder in der `scene-config.js` aktiviert wurde (vgl. Abb.11). Um die Einstellungen zu persistieren, können sie ebenfalls in den Optionen oder in der `Scene-config.js` angegeben werden. Diese Konfiguration ist ebenfalls austauschbar. Hier können auch die verschiedenen Interaktionsmöglichkeiten zur Animation und zu den Beschriftungselementen konfiguriert und getestet werden.

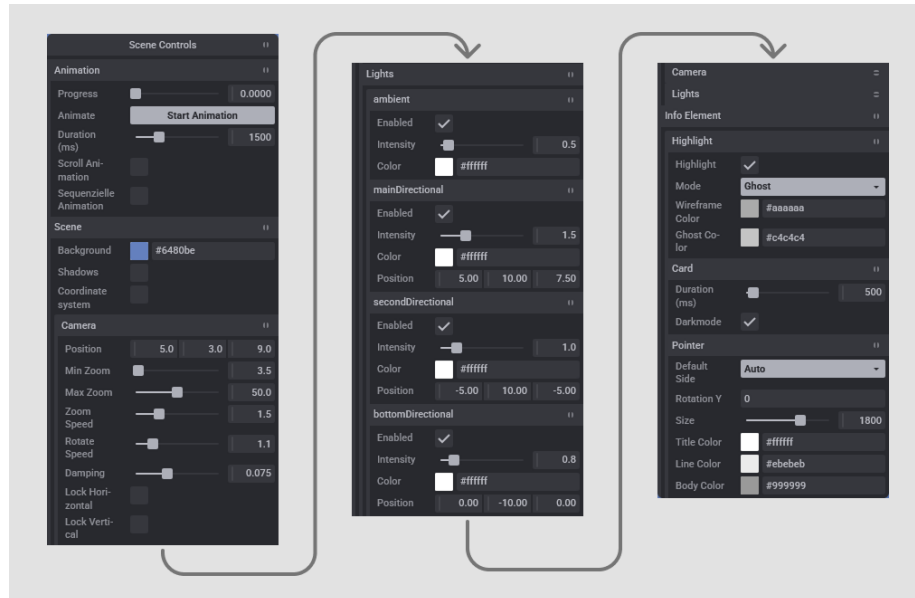


Abbildung 11: Parameter, welche Live in der Szene angepasst und erprobt werden können. Dieses UI Menü kann über `showDebugUI: true` aktiviert werden. Die Einstellungen sind in einer Ordnerstruktur organisiert und können übersichtlich verwaltet werden, diese Grafik stellt die verschiedenen Ordner aus Platzgründen nebeneinander dar.

Hervorhebungen Wenn ein 3D-Objekt ausgewählt wird, wird es Hervorgehoben. Dadurch kann die Aufmerksamkeit des Nutzers geleitet werden. Diese Hervorhebung ermöglicht zudem, kleine Elemente nach der Auswahl oder nachdem das Beschriftungselement betrachtet wurde, leichter wiederzufinden. Bei der Entwicklung wurden verschiedene Ansätze erprobt. Dabei hat sich der Modus `ghost` als beste Option erwiesen. Dieser reduziert standardmäßig die Materialien der anderen Objekte auf ein blasses Grau und verleiht ihnen einen leichten Transparenzeffekt (vgl. Abb. 12.1). Nutzer können jedoch verschiedene Modi wählen und diese passend zur Anwendung konfigurieren.

Beschriftungselemente: Beschriftungselemente ermöglichen die Erkundung des 3D-Modells und informieren den Nutzer interaktiv über die einzelnen Elemente. Diese Interaktion bietet einige Vorteile hinsichtlich der Nutzerwahrnehmung und des Nutzerverhaltens (vgl. Kellner, 2025d). Das Framework bietet 3D-, Teil-3D und 2D-Beschriftungselemente. 3D-Beschriftungselemente werden in der Szene am 3D-Modell platziert und zeigen auf das ausgewählte Objekt (vgl. Abb. 13). Diese Nähe unterstreicht den semantischen Zusammenhang und schöpft den 3D-Kontext voll aus.

Teil 3D- und 2D-Beschriftungselemente werden in einem Overlay über der 3D-Szene gerendert (vgl. Abb. 14). Der Unterschied liegt darin, dass die Teil-3D-Beschriftungselemente neben dem ausgewählten Objekt platziert werden und sich passend zum 3D-Objekt verschieben. 2D-Beschriftungselemente werden sta-

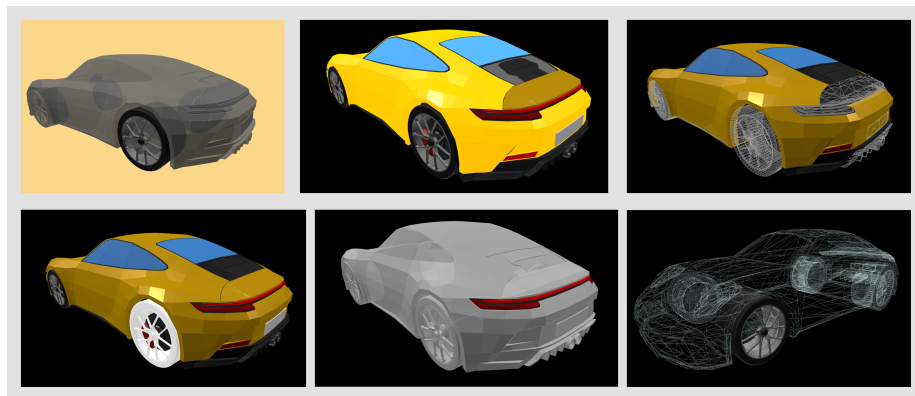


Abbildung 12: Auszug verschiedener erprobter Optionen zur Hervorhebung ausgewählter Objekte. Das Ausgewählte Objekt sollte nicht verfälscht werden, da die Szene dadurch weiter überladen wird und das betrachtete Element sich von der Realität abhebt. Sinnvolle Alternativen stellen Wireframes oder Ghost-Materialien für umliegende Objekte dar (vgl. Kellner, 2025d)

tisch am unteren Bildschirmrand angezeigt (vgl. Abb. 15).

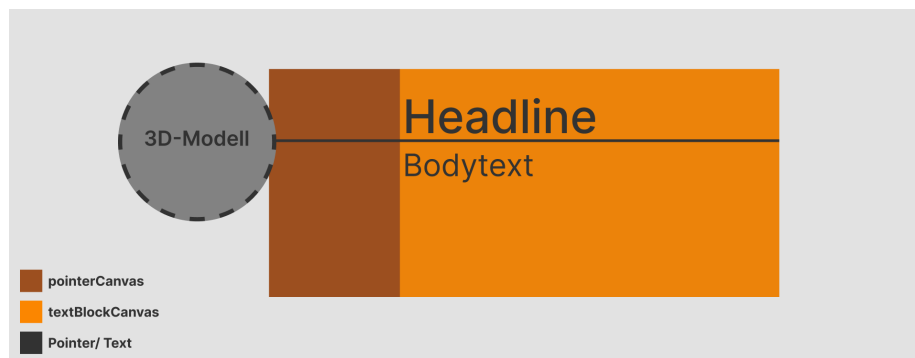


Abbildung 13: Aufbau Beschriftungselement pointer (vgl. Kellner, 2025d)

Formate und Standards: Bei der Konzeption und Entwicklung wurden diverse Industriestandards sowie weit verbreitete Frameworks und Formate verwendet (vgl. Kellner, 2025d). Die ausgewählten Technologien und Designentscheidungen ermöglichen eine Weiterentwicklung des Projekts und erlauben die Integration in bestehende Workflows im Kontext Web und 3D.

4.1.3 Bestehender Workflow

Um die Exploded-View-Animation auf einer Produktwebseite, Anleitungen oder anderen Webseiten zu integrieren, müssen einige Schritte durchlaufen werden. Vollständige Konfigurationen und Beispieldaten sind im GitHub-Repository hinterlegt (Kellner, 2025b)

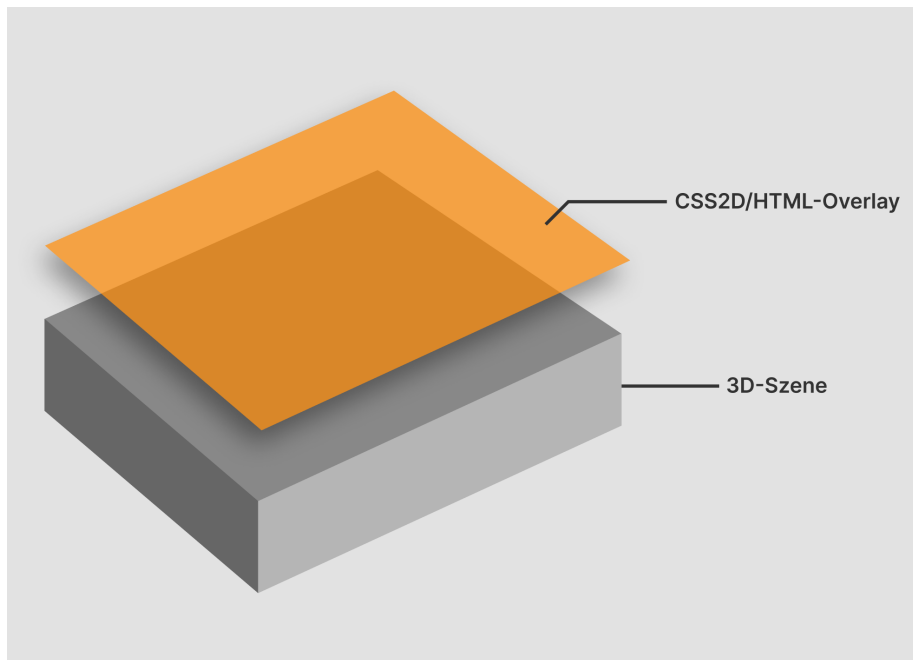


Abbildung 14: Visualisierung zum Overlay über der 3D-Szene (vgl. Kellner, 2025d)

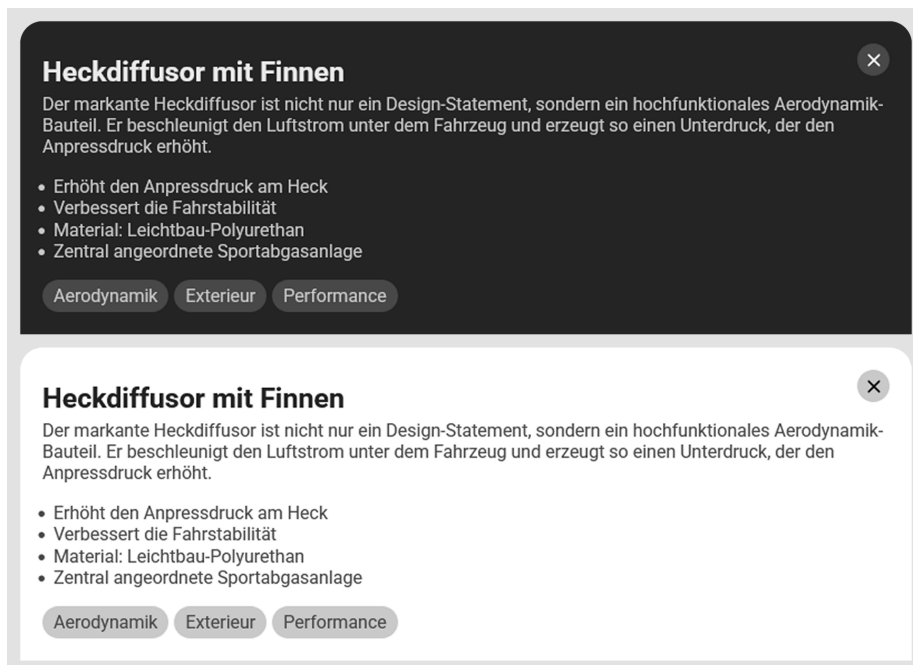


Abbildung 15: Cards, welche im Overlay platziert werden (vgl. Kellner, 2025d)

- **3D-Modell:** Das 3D-Modell muss erstellt und exportiert werden. Wichtig ist dabei, dass der Export des 3D-Modells im `.glb`-Format erfolgt. Dieses wurde aufgrund der effizienten Speicherung und der Kapselung weiterer Daten wie Texturen ausgewählt (vgl. Kellner, 2025d). Zudem sollte beim Export Y-Up ausgewählt werden, da die verwendete 3D-Umgebung Three.js die Y-Achse in vertikaler Ausrichtung hat. Dies ist konträr zu Programmen wie Blender, welche Z-Up verwenden. Zudem müssen alle Objekte einen einzigartigen Namen haben, dies wird jedoch bereits durch die meisten 3D-Programme garantiert. Objekte, die animiert werden, sollten dabei ein Objekt ohne Gruppierungen oder Sammlungen im 3D-Programm darstellen. Das Framework arbeitet für die Animationslogik primär mit `model.children` (vgl. Abb. 16). Verschachtelungen mit Parent-Child-Konstrukten sind möglich, jedoch fehleranfälliger.

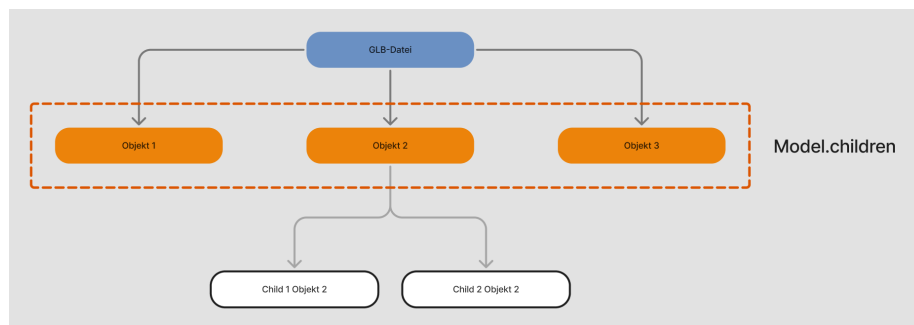


Abbildung 16: Visualisierung der Baumstruktur mit Eltern-Kind-Beziehungen im 3D-Kontext (vgl. Kellner, 2025d)

- **Installation:** Der Nutzer muss zunächst Zugriff auf die GitHub npm-registry erlangen, dafür wird ein Github Account und ein **personal access token** benötigt (vgl. GitHub Docs, n.d.). Diese registry ist nahtlos in GitHub integriert, ist jedoch für IDEs meistens nicht der Standard. Aufgrund dessen muss eine `.npmrc`-Datei im Hauptverzeichnis angelegt werden, die den Ort dieses NPM-Pakets beinhaltet.

npmrc-Datei:

`@maximiliankellner:registry=https://npm.pkg.github.com/`

Nun kann das NPM-Paket mit

`npm install @maximiliankellner/exploded-views` installiert werden.

- **Integration:** Nach der Installation kann das Paket in das bestehende Projekt integriert werden. Dafür muss ein Container, welcher die 3D-Szene beinhaltet, in den HTML-DOM eingefügt werden. Zudem muss das Paket und optional das Card-Styling (Abb. 15) im JavaScript importiert und konfiguriert werden.
- **Konfiguration:** Der Nutzer kann nun das Optionsobjekt anlegen. Die benötigten Dateien für das 3D-Modell, die Animation und optional für

Beschriftungselemente sowie ergänzende Szenen-Konfigurationen werden dort gebündelt (vgl. Listing 1). Alle möglichen Parameter und deren Standardwerte können in der `default-options.js` (Kellner, 2025a) überprüft werden.

- **Animation erstellen:** Der Nutzer definiert nun die Explosionslogik in einer separaten JSON-Datei. Zur Verbesserung des Workflows kann in der Debug-UI ein Koordinatensystem aktiviert werden. Zudem wird der Name des ausgewählten Objekts jeweils in der Browserkonsole ausgegeben. Mit diesen Informationen kann für jedes Objekt ein entsprechendes Objekt in der JSON-Datei erstellt werden. Dabei werden folgende Werte festgelegt.

Die wichtigsten Parameter:

- **level:** Ein Multiplikator für die Explosionsdistanz. Dieser Wert bestimmt, auf welcher Ebene sich das Objekt zum Abschluss der Animation befindet.
- **expDirection:** Ein Vektor $[X, Y, Z]$, der die Richtung der Bewegung angibt.
- **sequence** (optional): Definiert die Reihenfolge der Animation. Teile mit der gleichen Sequenznummer bewegen sich gleichzeitig.
- **speedMultiplier** (optional): Multipliziert die Geschwindigkeit in der die Animation dieses Objekts abgespielt wird, das Objekt erreicht den Endpunkt schneller.

Listing 2: Beispiel für eine `exp-config.json`

```
1  {
2    "_comment": "Die Richtung der Explosion (
3      expDirection) wird als Vektor [X, Y, Z]
4      angegeben.",
5    "objects": {
6      "Cube000": {
7        "level": 1,
8        "expDirection": [-1, -0.5, 0],
9        "sequence": 1
10       "speedMultiplier": 2.5
11     },
12     [...]
13   }
14 }
```

Nachdem diese Schritte erfolgt sind, kann mit der 3D-Szene interagiert werden und die Animation abgespielt werden.

4.2 Identifizierte Probleme und Defizite

Bei der Analyse und Anforderungsdefinition (3.2.1) wurden einige Probleme und Defizite festgestellt. Bereits bei der Installation sind Probleme aufgetreten, die in GitHub integrierte NPM-Registry ist zwar gut in den Entwicklungsprozess und in die Dokumentation eingebunden, verfügt jedoch über Sicherheitshürden wie Zugriffskontrollen, welche die Installation erschweren (vgl. 4.1.3). Ein Teil

der potenziellen Nutzer hatte Probleme, den Aufbau der Animation und entsprechend auch die Konfiguration zu verstehen. Die fehlende visuelle Vorschau hat dieses Problem weiter hervorgehoben, da Nutzer, insbesondere falls sie das System mit Ebenen und Vektoren nicht sofort verstanden haben, manuell einige Werte erproben mussten, um das Konzept zu verstehen. Neben der Konfiguration und Installation sollten auch weitere Verbesserungen vorgenommen werden. Steuerungselemente wie Buttons, Slider oder andere UI-basierte Interaktionsmöglichkeiten sollten besser integrierbar werden. Dadurch könnte das Paket besser in bestehende Webseiten und Designs eingebunden werden. Die aktuelle Implementation gibt im Normalfall UI-Elemente zur Steuerung der Animation vor. Falls diese nicht erwünscht sind, ist die aktuell vorgesehene Alternative, scrolling als Animationstrigger zu nutzen. Die Vor- und Nachteile der verschiedenen Interaktionsmöglichkeiten wurden bereits im Praxisprojekt erarbeitet (Kellner, 2025d). Ergänzend zu den UI-Elementen ist auch die Anpassung der 3D-Szene limitiert. Um den Konfigurationsaufwand für den Nutzer gering zu halten und einen schnellen Einstieg auch für weniger erfahrene Nutzer zu ermöglichen, wird Three.js vom Paket implementiert und kontrolliert. Diese Designentscheidung limitiert jedoch, in Kombination mit bestehenden JavaScript Limitationen, die Möglichkeiten, welcher der Nutzer hat, um die Eigenschaften der Szene zu bearbeiten. Zum Aktuellen Zeitpunkt ist der Nutzer bei der Implementation von Shadern oder komplexen Beleuchtungen limitiert. Weiteres Entwicklungspotenzial bieten Animationsfunktionen. Es könnten weitere Funktionen implementiert werden, welche Rotationseffekte, Skalierung oder neue Easing-Effekte zulassen, um komplexere und visuell ansprechendere Animationen zu ermöglichen. Dazu zählen auch Keyframes oder verbesserte Intervalle, die dem Nutzer mehr Freiraum bei der Konfiguration lassen. Auch bei diesen neuen Funktionen muss jedoch eine geringe Fehleranfälligkeit gewährleistet werden. Der aktuelle Ansatz über JSON-Dateien bietet einen strukturierten Ansatz, welcher menschen- und maschinenlesbar ist, jedoch können bereits minimale Abweichungen vom vorgesehenen Format zu Fehlern führen.

4.3 Auswirkungen auf Usability

Die genannten Limitationen und Defizite beeinflussen das Framework in verschiedenen Bereichen, die signifikanteste Problematik stellt jedoch aktuell die Nutzbarkeit dar. Die Konfiguration erfordert potenziell viele Iterationen, die JSON-Konfigurationen haben eine relativ hohe Fehleranfälligkeit und Anpassungen werden bei komplexen 3D-Modellen aufgrund der langen Konfigurationsdateien unübersichtlicher. Diese Faktoren resultieren in einer erhöhten Einstiegshürde und einer geringeren Motivation, das Framework zu verwenden.

5 Konzeption der Optimierungen und Erweiterungen

In diesem Kapitel werden mögliche Erweiterungen und Verbesserungen konzipiert, um die Nutzbarkeit und den Funktionsumfang des Pakets zu verbessern.

5.1 Designziele

Das primäre Designziel ist die Verbesserung der Nutzbarkeit. Der Workflow soll von der Installation bis zur konfigurierten und integrierten Animation geradliniger und einfacher verlaufen. Besonders die Konfiguration der Animation soll maßgeblich verbessert werden. Dafür soll ein visueller Editor entwickelt werden, da dieser den Prozess durch die Verwendung von UI-Patterns, Best Practices und bewährten Konzepten deutlich verbessern kann (vgl. 2.3). Der Editor soll auf einer dedizierten Webseite nutzbar sein, um die Konfiguration auch für Personen ohne Erfahrung in der Webentwicklung zu ermöglichen (Kellner, 2026a). Zudem soll er während der Entwicklung als Just-in-Time Hilfe lokal aktiviert werden können.

Ergänzend sollen weitere Animationsfunktionen implementiert werden. Diese sollen komplexere und visuell ansprechendere Animationen ermöglichen. Dabei soll ein signifikanter Anstieg der Komplexität bei der Konfiguration jedoch vermieden werden. Die Ladezeiten sollen weiterhin möglichst gering bleiben und die Erweiterungen sollen die Leistung nicht unverhältnismäßig beeinflussen. Dafür sollten bei der Implementierung der Funktionen stets Indikatoren wie RAM-Verbrauch, Latenz und FPS (Frames Per Second) überprüft werden. Die dafür benötigten Funktionen werden vom Betriebssystem oder Browser bereitgestellt oder sind bereits in der Anwendung implementiert.

5.2 Editor-Konzept

5.2.1 Rolle des Editors

Der Editor soll primär das Erstellen und Bearbeiten von Animationen und Konfigurationsdateien ermöglichen, da aktuell besonders die Erstkonfiguration der Animation problematisch ist (vgl. 4.2). Durch eine Vorschau soll das Animationskonzept (vgl. Abb. 8) besser verständlich und anwendbar werden. Zudem soll durch Konzepte wie direkte Manipulation und **What You See Is What You Get** (Myers, 1998, Shneiderman, 1983) eine bessere Nutzererfahrung und damit bessere Ergebnisse bei weniger Iterationen ermöglicht werden. Auch die Konfiguration der Szene soll nun nicht mehr nur mit verschiedenen Parametern erprobt, sondern auch direkt aus der UI Exportiert werden können.

5.2.2 Grundfunktionen des Editors

- **Visuelle Vorschau:** Um die Animationslogik greifbarer zu gestalten und eine bessere Übersicht über die Auswirkungen von Änderungen zu gewährleisten, sollte eine Live-Vorschau implementiert werden. Diese Vorschau muss den aktuellen Zustand der Animation abbilden, bei der Konfiguration muss das Endergebnis unabhängig vom aktuellen Animationsfortschritt angezeigt

werden. Die Live-Vorschau ermöglicht die Überprüfung der Änderungen, sodass Überlagerungen oder andere Visuelle Probleme in der Animation identifiziert und vermieden werden können. Zudem werden die Geschwindigkeit und Effizienz der Animationskonfiguration verbessert.

- **Objektselektion und Verschiebung:** Es soll eine Auswahlmöglichkeit für die einzelnen Komponenten des 3D-Modells implementiert werden. Nach der Auswahl sollte ein Objekt über bekannte UI-Muster verschoben und kontrolliert werden. Im 3D-Kontext wird Direct Manipulation meist in Form von Transformcontrols (vgl. 2.4.2) umgesetzt. Um Positionen und Richtungen standardisiert zu speichern, sollte auf Koordinaten und Vektoren zurückgegriffen werden.
- **Zeitmanagement der Animation:** Das Zeitmanagement ist besonders bei Animationen wichtig. Der Editor sollte ein zentrales Element implementieren, welches eine Übersicht über animierte Objekte und deren relevanten Intervalle im Animationsablauf visualisiert. Dadurch können Nutzer den Animationsablauf besser nachvollziehen und zeitgleiche Intervalle und Abfolgen schneller sowie übersichtlicher konfigurieren. Dabei könnte ein Timeline-Element, wie es auch in Videoschnittprogrammen oder anderer 3D-Software verwendet wird (vgl. 2.4.3), implementiert werden.
- **Parameter-Anpassung:** Im 3D-Kontext müssen viele Daten verwaltet und abgebildet werden, dazu zählen unter anderem Vektoren, Koordinaten, Rotationswerte, Farben, Objektnamen und Intervalle. Diese Daten müssen strukturiert mit einer hohen Informationsdichte abgebildet werden. Eine klare Hierarchie bei der Typografie und den UI-Elementen verbessert, in Kombination mit konsistenten Positionierungen und klaren Aufteilungen, die Übersichtlichkeit auch in komplexen Menüelementen. Zudem können Farben, Icons und **Microinteractions** zur Visualisierung des Systemstatus und von Fehlern verwendet werden (vgl. Kendrick, 2018).
- **Szenen-Parameter:** Die Konfiguration der 3D-Szene und der Interaktionen beeinflusst die Qualität des Endergebnisses maßgeblich. Das Framework muss anpassbar sein, sodass Nutzer die Szene an ihr bestehendes Design anpassen können. Es muss eine Möglichkeit geben, die Hintergrundfarbe anzupassen. Zudem müssen Nutzer verschiedene Beschriftungselemente erproben und wählen können. Diese Einstellungen ermöglichen gute Kontrastwerte und Anpassungen an das Designsystem. Zudem sollten auch Interaktionen mit der Szene einfach erprobt und konfiguriert werden können. Die Kameraposition und Ausrichtung sind dabei besonders wichtig. Jedoch können auch Zoom-Grenzen, Rotationssperren oder andere Parameter die Nutzererfahrung optimieren und Nutzer zu gewünschten Ansichten leiten.
- **Licht:** Licht spielt eine wichtige Rolle bei der visuellen Darstellung, es kann verwendet werden, um Elemente in Szene zu setzen oder die Stimmung der Szene anzupassen. Da verschiedene Render engines Lichter jedoch anders darstellen, ist es sinnvoll, die Lichtkonfiguration erst im Editor vorzunehmen (vgl. 2.2). Dabei sollten mindestens die Position, die Lichtfarbe, die Helligkeit sowie die Rotation abgedeckt werden.

- **Export der Dateien:** Der Editor sollte nicht nur als Vorschau dienen. Die Ergebnisse aus dem Editor sollten als Konfigurationsdateien im korrekten Format exportiert werden können. Dadurch kann neben der Erstellung auch eine Funktion zum bearbeiten implementiert werden.

5.2.3 Abgrenzung

Der Editor ist auf die einfache und effektive Konfiguration und Erstellung von Exploded Views mit dem entwickelten Paket spezialisiert. Es werden einige Funktionen zur Konfiguration der 3D-Szene sowie der 3D-Modelle und Lichter implementiert. Das Tool hat jedoch nicht den Anspruch oder das Ziel, eine vollwertige 3D-Modellierungssoftware zu ersetzen. Die Editor Komponente ist eine Erweiterung, welche den Workflow vereinfachen soll und wichtige Funktionen und Hilfen für die Erstellung von Exploded Views implementiert.

5.3 Erweiterte Animations- und Konfigurationsmöglichkeiten

5.3.1 Neue Optionen

- **Easing:** Aktuell verwendet das Paket standardmäßig verschiedene Easing-Effekte, Nutzer sollten in der Lage sein, aus vorgegebenen Effekten auszuwählen oder eigene Effekte zu erstellen.
- **Rotation:** Die Rotation von Objekten könnte Animationen visuell ansprechender gestalten und für einige Anwendungsfälle hohe Relevanz haben.
- **Sichtbarkeit:** Die Sichtbarkeit von Objekten könnte für einige Anwendungsfälle einen sinnvollen Animationsparameter darstellen. Dadurch könnten außenliegende, große Objekte versteckt werden, sodass innenliegende Objekte nicht verdeckt werden. problematisch ist jedoch die Umsetzung, da das Verstecken von Objekten über einfaches Ein- und Ausblenden den Animationsfluss stören würde. Zudem kann dieses Problem bereits über große Werte im `layer`-Parameter gelöst werden.
- **Kamera:** Die Kamera hat bereits eine Ladeanimation, es könnte jedoch eine Option für komplexere Kamerafahrten oder andere Ladeanimationen implementiert werden. Dabei ist es wichtig, dass diese Funktionalität die Komplexität nicht überproportional erhöht und intuitiv gestaltet wird. Problematisch ist, dass die Konfiguration einer Kamera das Kontrollieren der aktiven Kamera oder die Implementation einer zweiten Kamera erfordert.

5.3.2 Vereinfachungen

- **Intervalle:** Die Animationsparameter `sequence` und `speedMultiplier` könnten durch Intervalle ersetzt werden. Wenn jedes Objekt einen Animationsintervall hat, können immer noch aufeinanderfolgende Intervalle erstellt werden (vgl. `sequence`). Zudem kann über eine Verschiebung des Endpunkts der Effekt von `speedMultiplier` erzielt werden. Ergänzend würde diese Funktion auch andere Startpunkte und überlappende Intervalle ermöglichen.

- **Automatische Animationen:** Neben einer visuellen Konfiguration in einem Editor wäre eine Automatische Konfiguration hilfreich. Problematisch ist jedoch die Erstellung einer solchen Konfiguration, da das Volumen, der Objektsprung, die aktuelle Positionierung oder andere Daten kaum Grundlage für eine solche Berechnung bieten. Die komplexen Möglichkeiten im 3D-Kontext erschweren die algorithmische Erstellung einer Animation erheblich.

6 Umsetzung

6.1 Technische Architektur

Die Architektur ist relevant für die Wartbarkeit, Erweiterbarkeit und Performance des Projekts. Dafür hat die im Praxisprojekt erarbeitete Version des Frameworks bereits einige Paradigmen berücksichtigt. Das Framework wurde in Module aufgeteilt, diese Module haben klare Zuständigkeitsbereiche. Diese Modularisierung und Trennung der Verantwortlichkeiten ermöglichte eine reibungslose Weiterentwicklung und Integration der Editorfunktionen.

6.1.1 Integration des Editors in das Paket

Der Editor muss für die Vorschau der Animationen und der Szene viele Funktionen implementieren. Um doppelten Code zu vermeiden, welcher die Wartung und Erweiterung deutlich verkomplizieren würde, und um den Editor auch lokal als Just-in-Time Hilfe aktivieren zu können, wurden die Funktionen des Editors im Paket implementiert. Über die Konfiguration `editMode: true` kann der Editor aktiviert werden. Lediglich die Seite zum Onboarding wurde ausgelagert und ist somit nur auf einer dedizierten Webseite zu finden. Diese Erweiterung ermöglicht es auch Personen, welche nur für die Animation zuständig sind und das Paket nicht selbst implementieren, die Konfiguration zu erstellen oder zu bearbeiten und weiterzugeben, wie es bei einigen Personas der Fall ist Kellner, 2026a. Zudem ermöglicht diese Webseite, dass Nutzer vor der Integration in ihre Anwendung eine Animation erstellen können und dort entscheiden, ob das Framework die Anforderungen erfüllt.

6.1.2 Datenfluss und Events

Um trotz der großen Datenmenge und der zahlreichen Funktionen eine übersichtliche Struktur zu gewährleisten, wurden die Funktionen auf Module mit klar abgegrenzten Aufgabenbereichen aufgeteilt (Single Responsibility Principle, SRP). Der Datenaustausch erfolgt über den Import der verschiedenen Funktionen und Objekte. Dabei bildet ein Modul für den Zuständigkeitsbereich den einzigen relevanten Datenstand. Zudem werden die Konfigurationsdateien der aktuellen Sitzung zentral als Single Point of Truth (SPOT) verwendet. Zwischen den Modulen wird über Events und Callback-Funktionen kommuniziert (Lose Kopplung, Observer Pattern).

Es besteht jedoch auch Verbesserungspotenzial, beispielsweise ist die Event-basierte Kommunikation nicht flächendeckend implementiert. In folgenden Versionen könnte die Kommunikation mit Events auf alle Module und Verbindungen ausgeweitet, mit einem `EventBus` zentralisiert und klar dokumentiert werden, um die Wartbarkeit und Struktur weiter zu verbessern. Zudem ist bei der Entwicklung des Editors zwischenzeitlich eine `God-Class` entstanden. Der Funktionsumfang von `EditorController` ist während der Entwicklung stetig gewachsen und musste neu strukturiert werden. Dadurch sind die Klassen `EditorAnimationController` und `EditorLightController` entstanden, welche nun viele Funktionen kapseln und von `EditorController` delegiert werden.

6.1.3 Deployment- und Build-Prozess

In der Entwicklung hat GitHub eine große Rolle gespielt. Um die Änderungen und Erweiterungen zu dokumentieren und eine Zentrale Basis für Informationen zum Projekt zu schaffen, wurden die Git-commits auf Github mit weiterer Dokumentation und Links zu wichtigen Ressourcen ergänzt. Zu diesen Ressourcen zählt die NPM-Registry, welche für die Bereitstellung des Pakets verwendet wird (vgl. 4.2).

Neben der Versionierung des Codes ist auch das Bauen des Pakets eine wichtige Aufgabe, welche vom Build-Tool **Vite** übernommen wird. Es ermöglicht in der Entwicklung einen performanten Live-Server, der Anpassungen sofort abbildet und ermöglicht das konfigurieren und Bauen des NPM-Pakets, bevor dieses veröffentlicht wird.

In der öffentlichen Webversion wurde ein Onboarding-Prozess für das Paket entwickelt, dort wurde das Paket integriert. Um Änderungen schnell in der Produktionsumgebung zu deployen und testen zu können, wird ein GitLab Runner verwendet. Dieser kann Änderungen in eine CI/CD-Pipeline übernehmen, die Docker-Umgebung verwenden und den Container Bauen und auf dem Webserver integrieren. Die Zugriffe auf den Webserver werden über den Reverse Proxy Traefik gesteuert.

6.2 Implementierung des Editors

6.2.1 Konfigurationslogik

Die Konfigurationslogik greift auf zwei JSON-Dateien zurück. Dabei beinhaltet die `exp-config.json` weiterhin die Konfiguration für die Animation, es wurden jedoch einige Funktionen ersetzt oder neu hinzugefügt (vgl. 5.3). Die `scene-config.json` spielt nun auch wieder eine größere Rolle, da der Editor die Konfiguration der Lichter und Szene ermöglicht und Nutzer dadurch eher verleitet sind, die JSON-Datei anstelle der Optionen im `options`-Block des Konstruktors zu verwenden. Dieses Nutzungsverhalten beim Editor sollte aufgrund der implementierten Interaktionsmuster und UX-Verbesserungen eintreten (vgl. 5.2). Bei der Szenenkonfiguration muss jedoch berücksichtigt werden, dass der Editor primär für die Konfiguration der Animation gedacht ist und verschiedene Hervorhebungen sowie Beschriftungselemente zwar konfiguriert werden können, jedoch außerhalb des Editors erprobt werden sollten.

6.2.2 Übertragene UI-Konzepte aus Kreativsoftware

- **Grundlagen:** Der Designprozess wurde grob in vier Phasen aufgeteilt (vgl. 3.2). Die wichtigsten Ergebnisse aus diesem Prozess waren neben den Anforderungen und Personas (vgl. Kellner, 2026a) Designattribute, Farben und ein Klares System für Abstände (Abb. 17). Es konnten einige Designattribute und Konzepte festgehalten werden, welche das Interface ergänzend zu bestehenden Heuristiken ausmachen sollen.

klar, lesbar, professionell, einfach, intuitiv, konsistent, minimalistisch, anfängerfreundlich,

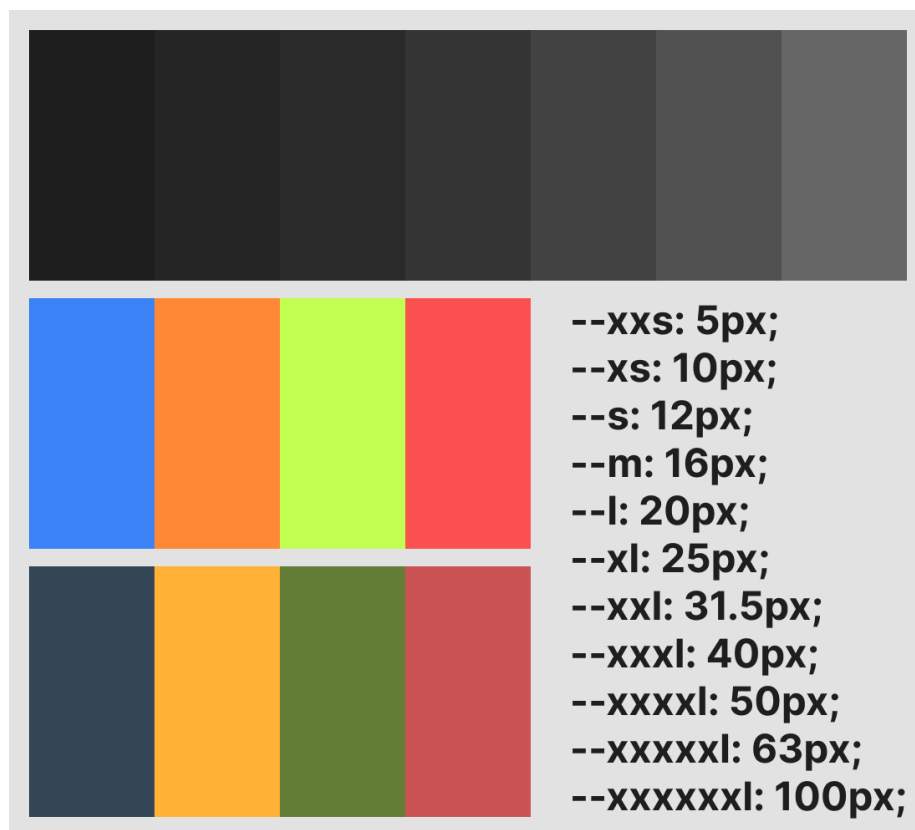


Abbildung 17: Ausgewählte Farben und das verwendete System zur Standardisierung der meisten Abstände und Distanzen

Minimal UI: Nur wichtige UI-Komponenten anzeigen, um den Fokus auf das 3D-Modell zu legen und Platz zu schaffen.

Feedback: Transparenz des Systemzustands, Vorausschau.

Diese Attribute wurden in den Komponenten umgesetzt und resultierten unter anderem in hoher Informationsdichte, klaren Hierarchien und Abgrenzungen durch farblich hervorgehobene Ränder, konsistente Abstände und farbliche Hervorhebungen wichtiger oder aktiver Elemente.

- **Layout:** Anstelle eines typischen U- oder L-Layouts, welches einen Großteil des Bildschirms für statische UI-Elemente reserviert und die Arbeitsfläche auf einen kleineren Bereich zwischen den Elementen beschränkt (vgl. Abb. 18), wurde für den Editor das Konzept der **Floating Menues** verwendet. Da dieser Editor sehr viel direkte Interaktion mit den 3D-Modellen auf der Arbeitsfläche erfordert und Entwickler den Editor in ihrer eigenen Web-Umgebung integriert aktivieren können, ist das Maximieren der Arbeitsfläche essenziell. Die UI-Komponenten werden nur aktiviert, falls sie benötigt werden, alternativ können sie vom Nutzer ein- und ausgeklappt oder minimiert werden.

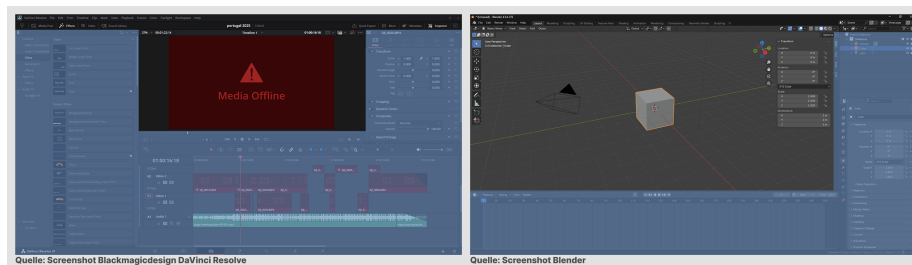


Abbildung 18: Die Arbeitsfläche oder der Vorschaubereich wird von U-Förmig oder L-Förmig angeordneten UI-Elementen umschlossen. Diese Anordnung verhindert, dass Inhalte überlagert werden, verkleinert den Arbeitsbereich jedoch maßgeblich.

- **Transform Controls:** Transform Controls konnten erfolgreich erprobt und implementiert werden. Sie erfüllen den erforderlichen Funktionsumfang (vgl. 2.4.2) und werden für die Bewegung und Rotation von Elementen verwendet. Die Transform Controls stehen als Teil der verwendeten 3D-Umgebung **Three.js** zur Verfügung (Three.js, n. d. b) und konnten mit geringen Anpassungen in das Paket implementiert werden.
- **Timeline:** Die entwickelte Timeline-Komponente erfüllt ebenfalls bekannte Funktionen und Merkmale (vgl. 2.4.3). Dabei werden alle animierten Objekte in Verbindung mit dem Intervall, in dem die Animation stattfindet, abgebildet. Zudem gibt das Element die Anzahl der animierbaren Objekte an und ermöglicht die Steuerung der Animation und der Animationsdauer. Über einen Scrubber kann präzise ein bestimmter Zeitpunkt der Animation ausgewählt werden und die Komponente kann minimiert

werden, sodass nur die wichtigsten Funktionen aktiv bleiben. Eine weitere Besonderheit dieser Timeline ist, dass sie zwischen 0 und 100 Prozent verläuft. Dadurch kann die Animationsdauer einfach skaliert werden und ist nicht durch Bildwiederholraten oder ähnliches verzerrt oder beschränkt.

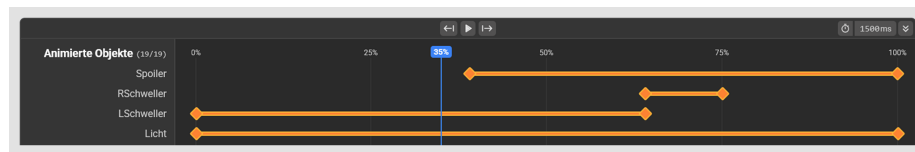


Abbildung 19: Implementierte Timeline-Komponente mit einigen konfigurierten Intervallen

- **Keyframes:** Für die neue Animationslogik hat ein Objekt zwei Keyframes. Über diese kann der Ausgangszustand und der Endzustand im Animationsverlauf verschoben werden. Diese Keyframes bilden das Intervall in dem das Objekt bewegt wird und werden in der Timeline durch orangefarbene Elemente hervorgehoben (vgl. Abb. 19). Der Start und Endpunkt des Intervalls kann in einem Menü (vgl. Abb. 20.1) angepasst oder in der Timeline durch das Verschieben der Keyframes verschoben werden. Beim Verschieben in der Timeline erhält der Nutzer eine direkte Vorschau in der 3D-Szene zum Zielzeitpunkt, da der Scrubber der Timeline sich mit den Keyframe Griffen mitbewegt. Diese Daten fließen ebenfalls in die zentrale Konfiguration ein. Dadurch, dass alle Objekte in der Timeline aufgelistet sind, können Nutzer die Keyframes verschiedener Objekte schnell vergleichen und Intervalle abstimmen.



Abbildung 20: Editor Panels mit verschiedenen Parametern zur Konfiguration von Lichtern oder Elementen

- **Vorschau:** Neben der Vorschau des aktuellen Animationszustands kann auch der Endzustand visualisiert werden. Diese Vorschau wird verwendet, sodass der Nutzer unabhängig vom aktuellen Animationsfortschritt die Konfiguration tätigen kann. Dabei wird auf eine Darstellung im Wireframe

Stil zurückgegriffen (vgl. Abb. 21.1). Diese ermöglicht Überlagerungen des Vorschauobjekts und des Ursprungsobjekts und bildet die Form und die Ausmaße des Ursprünglichen Objekts klar ab (vgl. 2.4.5). Parallel zu den Objekten in der Szene kann auch die Szene selbst konfiguriert werden, dafür stehen diverse Optionen zur Verfügung, welche zu großen Teilen auch in Echtzeit im Editor angepasst werden können, diese haben ebenfalls Auswirkungen auf die Szenenkonfigurationsdatei und können exportiert werden

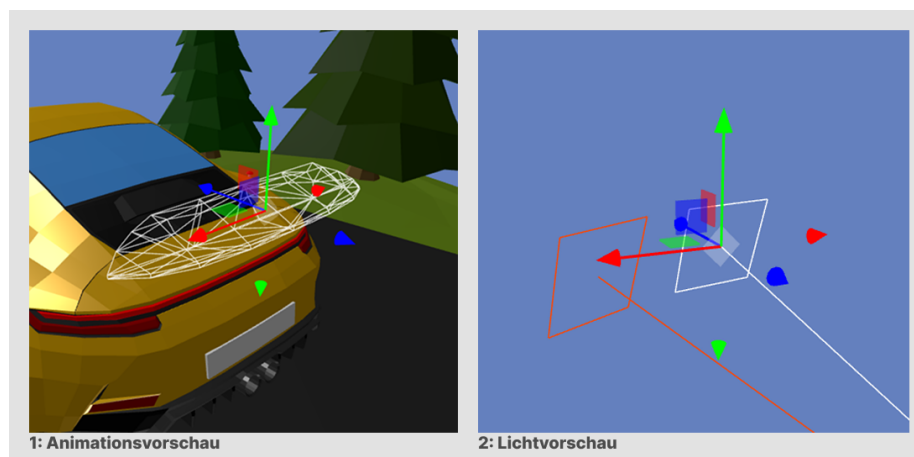


Abbildung 21: 1: Wireframe Objekt, welches ein Duplikat des Ursprungsobjektes ist und mit einem Wireframe Material an der Endposition der Animation platziert wird.

2: DirectionalLightHelper, welcher zur Vorschau der Lichtfarbe, des Targets, der Größe und der Position implementiert wurde.

- **Lichter:** Um die Lichter umfangreich konfigurieren zu können wurde ebenfalls ein **Floating Menu** implementiert (vgl. Abb. 20.2). Dabei können die wichtigsten Konfigurationen kompakt konfiguriert werden. Zudem werden Lichter, welche nicht vom Typ **ambient** sind, mit einem Light-Helper (vgl. Three.js, n. d. a) versehen, dieses Vorschaulement ermöglicht es die Position des Lichts, das Target, die Rotation und die Lichtfarbe intuitiv abzulesen und ermöglicht auch bei vielen Lichtern eine gute Übersicht (vgl. Abb. 21.2). Diese Light-Helper können ebenfalls intuitiv über Transform Controls manipuliert werden. In weiteren Versionen könnten mehr Optionen und Lichttypen implementiert werden um den Funktionsumfang weiter zu steigern. Ausgewählte Parameter werden ebenfalls in der entsprechenden Auflistung der Lichter abgebildet.
- **Komprimierte Listen:** Um eine zentrale Übersicht über alle Elemente und viele Parameter zu bekommen wurde ein Menü mit verschiedenen Tabs erstellt. Die verschiedenen Tabs unterteilen die Daten in die Kategorien Szene, Objekte und Lichter. Innerhalb der Tabs wurden die Daten, Objekte und Parameter komprimiert in Listen angeordnet und entweder in Ordern (vgl. Abb. 22.1) organisiert oder mit Suchfeldern versehen (vgl.

Abb. 22.2). Das Hinzufügen von Lichtern und das Exportieren von Konfigurationsdateien wird ebenfalls in diesem Element ermöglicht. Komprimierte Listen konnten also sinnvoll integriert werden, wurden jedoch deutlich durch Ordner, Tabs und Parameter- und Status-Vorschau ergänzt.

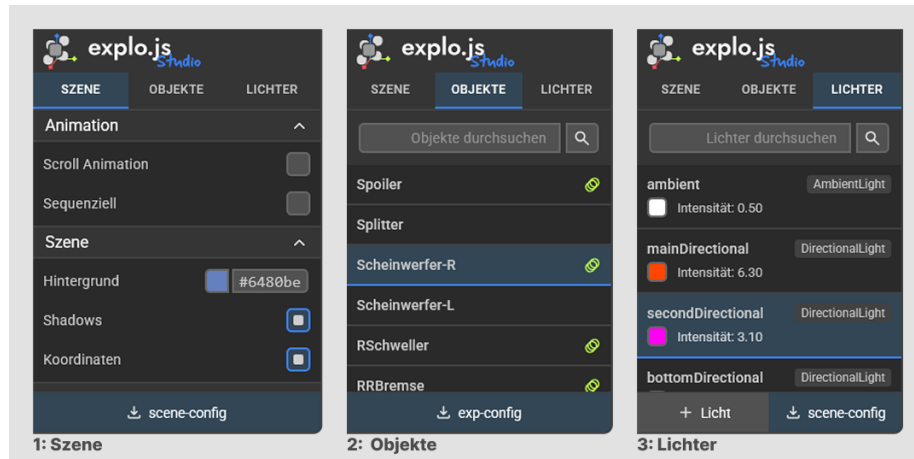


Abbildung 22: Zentrales Objektmanagement. Hier werden Lichter, Objekte und Szenenparameter gesammelt und bearbeitet. Ergänzend können hier Lichter erstellt und Konfigurationsdateien exportiert werden.

Schließlich ist festzustellen, dass viele UI- und Interaktions-Konzepte aus bestehender Kreativsoftware sinnvoll integriert werden konnten und mit einigen Anpassungen den Editor bereichern, indem Nutzer bekannte Muster und Funktionen wiedererkennen. Die Konzepte konnten jedoch so angepasst und integriert werden, dass sie keine Einschränkungen der Nutzbarkeit oder des Funktionsumfangs verursachen.

6.3 Erweiterte Animationslogik

Neben der vereinfachten Konfiguration durch den Editor und entsprechender UI-Elemente konnte auch der Funktionsumfang gesteigert werden.

6.3.1 Intervalle

Die Parameter `speedMultiplier` und `sequence` konnten durch Keyframes ersetzt werden. Mit den Parametern `start` und `end` kann der Start- und Endpunkt des Animationsintervalls präzise festgelegt werden. Nach dieser Änderung können die gleichen Ergebnisse erzielt werden. Ergänzend sind jedoch auch überlappende Animationen und weitere zugeschnittene Ergebnisse möglich. Diese Implementation ist deutlich intuitiver und besonders in Kombination mit der Timeline-Komponente (vgl. 6.2.2) übersichtlich dargestellt.

6.3.2 Easing

Um die Animationen flüssiger zu gestalten und sie harmonischer in bestehende Design-Konzepte integrieren zu können, wurden verschiedene Easing-Effekte implementiert. Dadurch können Animationen hochwertiger, flüssiger, dynamischer oder verspielter wirken. Dabei wurde bereits auf die Funktionen aus dem Paket Anime.js zurückgegriffen (Kellner, 2025d), nun werden jedoch auch weitere Easing-Effekte implementiert. Nutzer können dabei die vorgegebenen Effekte in der UI (vgl. Abb. 22.1) oder neue Effekte aus der interaktiven Dokumentation von Anime.js auswählen (vgl. Garnier, 2025).

6.3.3 Rotation

Nachdem die Komplexität durch die Intervalle zur Animationssteuerung reduziert werden konnte, wurde ein neuer Parameter eingefügt. Es können ein X-, Y- und Z-Wert in Grad angegeben werden. Dadurch kann das Objekt rotiert werden. Diese Konfiguration erhöht die Komplexität für den Nutzer kaum, kann das Endergebnis jedoch maßgeblich verbessern, da deutlich dynamischere und umfangreichere Animationen möglich werden. Ebenfalls können Kontextbezogen sinnvolle Animationen ermöglicht werden (vgl. Abb.23). Auch ein erhöhter Detailgrad und Feinschliff, wie beispielsweise rotierende Schrauben, können die Qualität der Animation erhöhen.

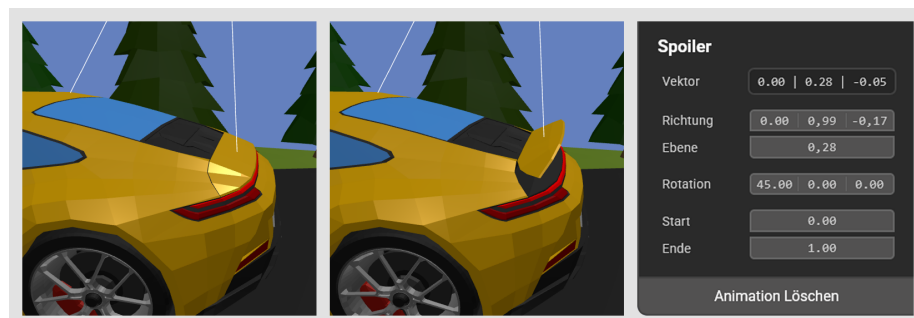


Abbildung 23: Beispielkonfiguration Spoiler. Der Heckspoiler des Fahrzeugs rotiert und kann so im aktiven Zustand dargestellt werden.

6.3.4 Kamera

Es wurden keine vom Nutzer konfigurierbaren Kamerafahrten eingebaut. Kamerafahrten, beispielsweise mit Bézierkurven, würden zwar den Funktionsumfang steigern, bringen jedoch nur geringfügige Mehrwerte, da bei diesem Paket neben der Animation auch die Interaktion und die eigenständige Erkundung durch den Nutzer im Vordergrund steht. Zudem würde die Integration den Entwicklungsaufwand und die Komplexität für Nutzer des Pakets unproportional erhöhen. Es konnte kein System zur Abstraktion von Kamerafahrten, wie es beispielsweise für Exploded Views konzipiert wurde, entwickelt werden. Aufgrund dessen wird weiterhin die Ladeanimation mit Look-At-Logik verwendet.

6.3.5 Lichter

Auf eine Animation der Lichter wurde ebenfalls verzichtet. Es ist jedoch denkbar, dass diese Funktionalität in folgenden Versionen nachgereicht wird. Durch gezielte Änderungen in der Benutzeroberfläche könnten diese Funktionen, insbesondere in Kombination mit der implementierten Look-At-Logik, sinnvoll integriert werden und den Funktionsumfang des Pakets sinnvoll ergänzen.

6.4 Beispiel-Workflows

Nachdem der bestehende Workflow in Kapitel 4.1.3 untersucht wurde, wird nun der neue Prozess mit den getroffenen Designentscheidungen analysiert.

6.4.1 Allgemeiner Workflow

- **3D-Modell:** Der Prozess bei der Erstellung und Vorbereitung des 3D-Modells bleibt weitestgehend unverändert (vgl. 4.1.3). Die Benennung der Komponenten hat jedoch weiter an Relevanz verloren, da Nutzer Objekte nun visuell identifizieren, auswählen und konfigurieren können.
- **Installation:** Die Installation konnte signifikant erleichtert werden. Es sind keine `.npmrc`-Dateien oder andere Konfigurationen notwendig. Nutzer können das Paket von `npmjs.com` installieren, indem sie `npm i explo.js` ausführen.
- **Integration:** Die Integration bleibt unverändert. Nutzer können das Paket weiterhin mit wenigen Zeilen Code integrieren. Es muss jedoch ebenfalls der aktualisierte Paketname verwendet werden

```
import ExplodedViewer from 'explo.js';
```

Optional kann weiterhin das Standarddesign für die Beschriftungselemente importiert werden.

```
import 'explo.js/default-card-style.css';
```

- **Konfiguration:** Die Konfiguration verwendet weiterhin ein Konfigurationsobjekt mit Optionen und dem verwendeten Container, welcher die Animation beinhalten soll. In den Optionen müssen weiterhin relevante Pfade zum 3D-Modell und der Animationskonfiguration abgelegt werden. Alle weiteren Pfade und Konfigurationen sind optional, können jedoch, wie beispielsweise bei der Auswahl und Konfiguration der Easing-Effekte, einen größeren Funktionsumfang bieten (vgl. 6.3.2). Zudem sollten Nutzer hier den Editmode aktivieren, um die Animation oder Szene zu konfigurieren.
- **Animation erstellen:** Die Animation basiert weiterhin auf einer JSON-Datei. Der Workflow wurde jedoch maßgeblich durch den Editmode erleichtert. Nutzer können in der Konfiguration den Editmode über `editMode: true`, aktivieren und im Visuellen Editor die Konfiguration unter Berücksichtigung

diverser Heuristiken, UI-Muster und Interaktionsmuster intuitiv umsetzen. Um Formatfehler zu vermeiden, sollten Nutzer idealerweise auf die Konfiguration in der JSON-Datei verzichten.

Die wichtigsten Parameter:

- **level**: Ein Multiplikator für die Explosionsdistanz. Dieser Wert bestimmt, auf welcher Ebene sich das Objekt zum Abschluss der Animation befindet.
- **expDirection**: Ein Vektor $[X, Y, Z]$, der die Richtung der Bewegung angibt.
- **rotation**: Ein Tupel, welcher Gradzahlen beinhaltet. Es kann eine Rotation über die X-, Y- und Z-Achsen konfiguriert werden.
- **start**: Der Startpunkt der Animation, ($0 \leq \text{start} < 1$). Der Startwert muss zudem kleiner als der Endwert sein und stellt einen Prozentwert im Animationsverlauf dar.
- **end**: Der Endpunkt der Animation ($0 < \text{end} \leq 1$). Der Endwert muss zudem größer als der Startwert sein und stellt einen Prozentwert im Animationsverlauf dar.

Listing 3: Beispiel für eine neue `exp-config.json`

```
1  {
2    "_comment": "Vektoren und andere Wertepaare werden
3      in der Reihenfolge [X, Y, Z] angegeben.",
4    "objects": {
5      "Spoiler": {
6        "level": 0.25,
7        "expDirection": [
8          0,
9          0.99,
10         -0.12
11       ],
12       "rotation": [
13         45,
14         0,
15         0
16       ],
17       "start": 0,
18       "end": 1
19     }
20   }
21 }
```

- **Szene konfigurieren (Optional)**: Die Konfiguration der Szene ist nun ebenfalls im Editor möglich, ist dem Testen mit der Debug-UI (vgl. Abb. 11) jedoch aufgrund der Export-Funktion deutlich überlegen. Nutzer sehen weiterhin eine Vorschau der Optionen, können die Lichter und Szenenparameter jedoch nun, gebündelt in einer JSON-Datei, aus dem Editor exportieren. Die Debug-UI wurde jedoch nicht entfernt, da sie die Möglichkeit

bietet außerhalb des Editmodes diverse Einstellungen zu erproben und besonders für die Beschriftungselemente einen Mehrwert bietet.

- **Lichter konfigurieren (Optional):** Lichter können nun effektiv konfiguriert werden. Es gibt weiterhin eine Standardkonfiguration, zusätzlich können Lichter nun im Editor einfacher konfiguriert, erstellt und entfernt werden. Zudem wurden weitere Lichttypen ergänzt (vgl. 6.2.2). Diese Daten werden mit der Szenenkonfiguration gebündelt.

6.4.2 Onboarding

Neben der lokalen Version wurde auch ein Onboarding-Prozess erstellt, welcher besonders für neue Nutzer und das Erstellen von Animationen Vorteile bietet. Dabei können Nutzer auf einer Webseite ihr 3D-Modell und optional auch bestehende Konfigurationsdateien hochladen. Zudem werden hier Links zu weiteren Ressourcen zur Erkundung oder Hilfestellungen geboten. Dieser Schritt ist zuständig für die Verifikation des Dateiformats und Inhalts. Diese dedizierte Webseite ermöglicht also das Erstellen, Bearbeiten und Überprüfen von Animationen und Konfigurationsdateien. Zudem deckt sie einige Anwendungsfälle ab, in denen Aufgabenteilung und die Austauschbarkeit der Dateien eine große Rolle spielt (Kellner, 2026a). Wie in Kapitel 2.3 erarbeitet, können umfangreiche Onboarding-Prozesse die Nutzerzufriedenheit senken. Aufgrund dessen wurde der Import-Prozess in die Bereiche Animation erstellen und Animation Bearbeiten aufgeteilt (vgl. Abb.24). Daraufgehend werden Nutzer über Probleme mit den Dateien informiert oder direkt in den Editor weitergeleitet. Im Editor können dann Konfigurationen erstellt und exportiert werden. Anstelle eines umfangreichen Onboardings kann der Nutzer Szenenparameter und andere Einstellungen in verschiedenen dafür vorgesehenen Bereichen tätigen. Besonders für wiederkehrende Nutzer ist dieser Ansatz effizienter, da sie nicht von einer linearen Einführung verlangsamt werden. Dieses **Progressive Disclosure** (Nielsen, 2006) und die **Just-in-Time Hilfen** werden auch innerhalb anderer UI-Elemente zur Strukturierung und Übersicht verwendet (vgl. Abb. 20, 22).

Abgrenzung: Die Webseite wurde im Rahmen der Arbeit erstellt und veröffentlicht, wird jedoch möglicherweise nicht dauerhaft zugänglich bleiben. Dies ist ein weiterer Grund für die Designentscheidung, dass der Editor auch lokal im Paket verfügbar ist.

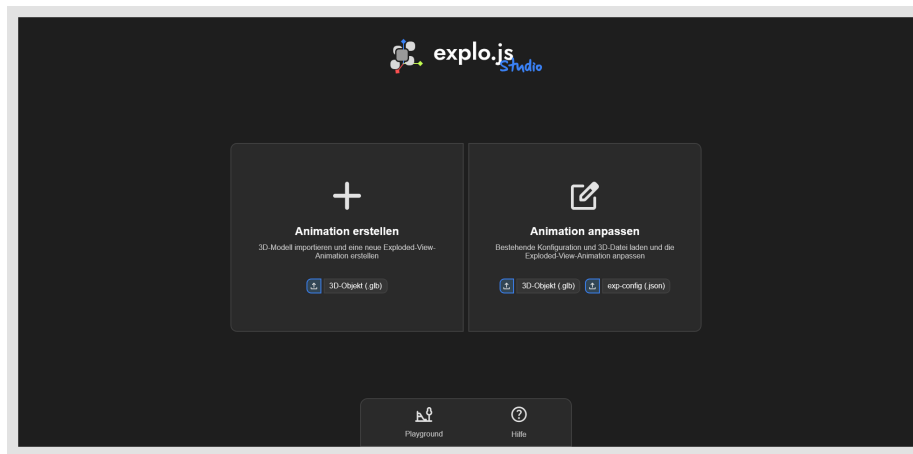


Abbildung 24: Onboarding landingpage (Quelle: exploded-views-kellner.de-fender.it)

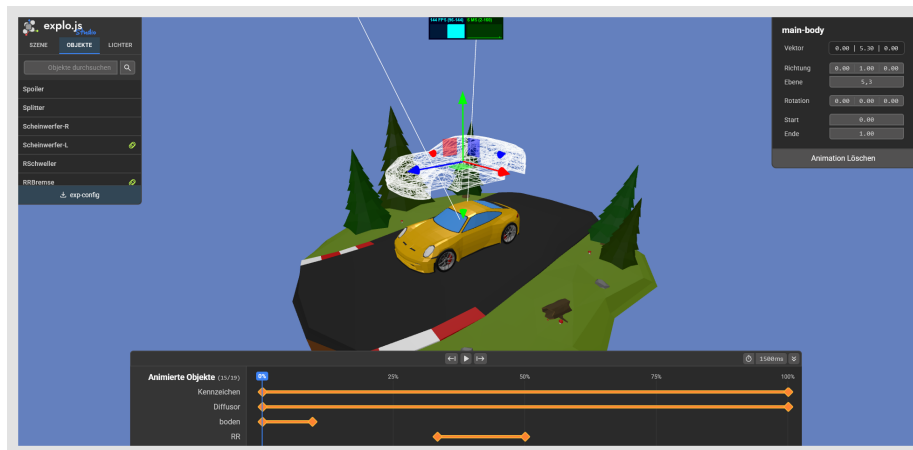


Abbildung 25: Editor mit konfiguriertem 3D-Modell und Szene (Quelle: exploded-views-kellner.de-fender.it)

7 Ergebnisse und Diskussion

7.1 Validierung der Effizienzsteigerung

7.1.1 Versuchsaufbau

Um die verbesserte Konfigurationsgeschwindigkeit und Nutzererfahrung zu Validieren wurde ein Versuch durchgeführt. Als Basis diente ein 3D-Modell, welches in Blender erstellt wurde. Dazu wurde ein Zielzustand konzipiert, welcher nach Abschluss der Animation erreicht werden sollte. Das 3D-Modell ist eine Pyramide, bestehend aus 5 Einzelteilen (vgl. Abb. 26).

7.1.2 Methodik

Es wurden zwei Durchläufe getätigt. Dabei wurde jeweils mit der .glb-Datei in einer unkonfigurierten Szene gestartet und die Konfiguration schrittweise erstellt und getestet. Die Zeit wurde gestoppt, sobald das Animationsziel (vgl. Abb. 26) erreicht werden konnte und die Konfiguration eingebunden war.

- **Durchgang A (manuelle Konfiguration):** Die Komponenten des 3D-Modells wurden umbenannt, sodass sie besser zuzuordnen sind, dann wurde die leere exp-config.json Datei auf Basis der Installationsanleitung erstellt und mit fünf leeren Objekten gefüllt. Anschließend wurden die Namen, Vektoren und Ebenen der JSON-Objekte angepasst und erprobt. Dabei mussten die Richtungen für die horizontale Bewegung der mittleren Elemente mehrfach angepasst werden, da ohne den Editmode die Orientierung im dreidimensionalen Raum maßgeblich erschwert wird. Nach einer finalen Validierung konnte die Zeit gestoppt werden.
- **Durchgang B (Konfiguration im Editmode):** Bis auf das 3D-Modell wurde ein leerer Optionen Block verwendet. Der Editmode wurde aktiviert und die Konfiguration konnte innerhalb des visuellen Editors vorgenommen werden. Es wurden Transform-Controls zur groben Positionierung verwendet, anschließend wurden die Parameter über die UI-Komponenten präzisiert. Nach einer finalen Validierung konnte die Zeit gestoppt werden.

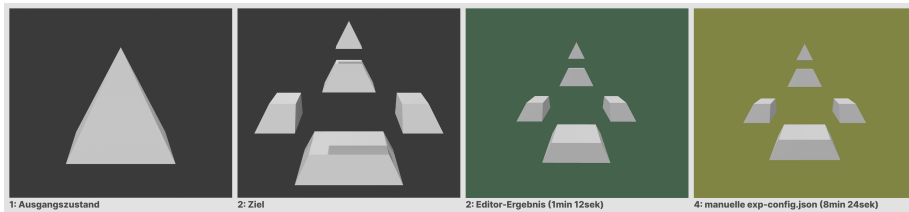


Abbildung 26: 3D-Modell Pyramide im Ausgangs (1) und Zielzustand (2). Zudem die erzielten Ergebnisse, welche mit und Ohne Editmode erzielt werden konnten.

7.1.3 Daten und Analyse

Der Test konnte einen gravierenden Unterschied in der Konfigurationsgeschwindigkeit und der Nutzererfahrung feststellen. Während der manuelle Konfiguration 504sek in Anspruch nahm konnte die Konfiguration im Editmode in 72sek vorgenommen werden. Die Konfigurationsgeschwindigkeit wurde in diesem Test also um den Faktor sieben verbessert. Dabei ist zu beachten, dass es sich um ein 3D-Modell mit vergleichsweise geringer Komplexität handelt. Zudem wurden nur die Parameter `exp-direction` und `level` verwendet. Wenn Intervalle, Rotationen und Szenenkonfigurationen gewählt werden steigt der Konfigurationsaufwand, die Komplexität und damit der Zeitunterschied weiter an. Die manuelle Konfiguration ist von Fehlversuchen und bloßen Zahlen geprägt, zudem erfolgt das Konfigurieren in einem JSON-Editor. Dadurch sind viele Kontextwechsel zwischen der Animation und der JSON-Datei erforderlich, diese können den Nutzer ermüden (vgl. Rubinstein et al., 2001).

Der Editmode bündelt alle Funktionen in eine Umgebung, zudem werden Funktionen durch gekapselte UI-Elemente, klare Hierarchisierung und Progressive Disclosure weiter voneinander abgegrenzt. Auch die Identifikation der Objekte wird erleichtert, da der Editor eine visuelle Identifikation der 3D-Komponenten und eine Suchfunktion bietet. Zudem visualisieren die Transform-Controls die Ausrichtung im 3D-Raum und verringern so die Fehlerquote maßgeblich. Dadurch ist die Nutzererfahrung besser und die Frustration fällt geringer aus.

7.2 Ergebnisse

Diese Arbeit hat diverse Verbesserungen am bestehenden Framework und neue Erkenntnisse hervorgebracht. Das Framework konnte von einem groben Prototypen zu einer Version mit besserer Nutzbarkeit und einem größeren Funktionsumfang weiterentwickelt werden. Bei der Konzeption und Entwicklung der Verbesserungen konnte die Vorarbeit validiert werden. Der bestehende Code konnte problemlos erweitert werden und erforderte nur geringfügige Anpassungen. Aufgrund dessen wurde die Konzeption und Entwicklung der neuen Komponenten und Funktionen in einem ähnlichen Rahmen umgesetzt.

Abschließend ist ein NPM-Paket entstanden, welches Exploded View Animationen im Web erstellen kann. Dabei liegt der Fokus auf einfacher Erstellung und Integration der Animationen, indem komplexe Animationen durch UI-Elemente und intuitive Interaktionsmuster abstrahiert werden. Diese Funktionen werden vom umfangreichen Editor übernommen, welcher die Konfiguration maßgeblich verbessert hat. Dadurch senkt das Framework die Einstiegshürde für interaktive 3D-Elemente auf Webseiten und die Erfahrung für den Endnutzer kann schöner, interaktiver und spannender gestaltet werden.

7.3 Bewertung

Die untersuchten Funktionen, Interaktions- und UI-Muster konnten sinnvoll integriert werden und das Projekt bereichern. Der Editmode hat die Einstiegshürde und den allgemeinen Workflow stark verbessern können. Ebenfalls konnte der Funktionsumfang sinnvoll erweitert werden, sodass bei sinkender Komplexität

der Funktionsumfang gesteigert werden konnte. Es konnten auch weitere Zielgruppen, welche durch Personas modelliert wurden, angesprochen werden. Das Erstellen der Animation ist durch die Konfigurationswebseite nun auch ohne Qualifikationen in der Webentwicklung möglich. Diese Arbeitsteilung und die Kompatibilität mit verschiedenen Workflows wird weiter durch die Austauschbarkeit der Konfigurationsdateien unterstützt. Trotz der umfangreichen Erweiterungen konnte auf einen signifikanten Anstieg von Fremdcode verzichtet werden. Es wurde sich größtenteils auf Eigenentwicklungen und die bereits ausgewählten Technologien verlassen.

7.4 Limitationen

Konzeptionell hat der Editor nicht den Anspruch oder das Ziel, eine vollwertige 3D-Modellierungssoftware zu ersetzen. Die Editor-Komponente ist eine Erweiterung, welche den Workflow vereinfachen soll und wichtige Funktionen und Hilfen für die Erstellung von Exploded Views implementiert.

Weiteres Verbesserungspotenzial besteht bei der Codequalität. Bei der Entwicklung wurde Rücksicht auf einige Konzepte, Paradigmen und Heuristiken genommen, es sind jedoch noch Verbesserungen, beispielsweise bei der Entkopplung, möglich. Nicht alle Module sind vollständig entkoppelt. Ziel weiterer Entwicklungen könnte eine vollständige Entkopplung und Zentralisierung der Events sein.

Auch die Bewertung der Usability wurde anhand weniger Personen durchgeführt. Es wurde sich mit Personen aus dem Fachgebiet Informatik ausgetauscht und es wurden Personas sowie eigenständige Bewertungen und Analysen verwendet. Dieser Prozess ist relativ subjektiv und entstand nur unter Einbezug einer kleinen Zielgruppe. Ergänzende Perspektiven durch umfangreiche Nutzertests oder Analyse-Tools in der Webumgebung könnten neue und objektivere Ergebnisse liefern.

8 Fazit und Ausblick

8.1 Zusammenfassende Beantwortung der Forschungsfragen

- **F1: Welche zentralen Probleme und Defizite von Explo.js beeinflussen den Workflow und die Nutzbarkeit, und wie lassen sich diese durch gezielte konzeptionelle und technische Anpassungen verbessern?** Das Hauptproblem des Pakets bestand in der Installation und Konfiguration. Nachdem das NPM-Paket in die offizielle NPM-Registry migriert und ein Editor sowie eine Webseite mit Onboarding und Editor erstellt wurden, konnten diese Probleme gelöst werden. Zudem konnten gezielte Anpassungen und Erweiterungen die Animations- und Interaktionsmöglichkeiten, auch für fortgeschrittenere Entwickler, erweitern. Dadurch ist eine schnellere, einfachere und bessere Integration in andere Projekte möglich.
- **F2: Welche Visualisierungs- und Interaktionsmuster aus etablierter 3D-Software lassen sich sinnvoll auf einen webbasierten Exploded-View-Editor übertragen, um die Konfiguration, Steuerung und Vorschau von Animationen zu verbessern?**

Die Konzeption umfasste eine intensive Analyse bestehender Softwareprodukte für die Medienerstellung und Gestaltung. Dabei konnten diverse UI- und Interaktionsmuster identifiziert werden. Besonders UI-Muster wie die Timelinekomponente, Transform Controls und Hervorhebungen durch Wireframes und Bounding Boxes sind bekannte Muster, welche erfolgreich adaptiert und implementiert werden konnten. Die Verwendung dieser Muster verbessert die Geschwindigkeit, mit der die Nutzer die UI verwenden können. Zudem stellen sie wichtige und erprobte Konzepte sowie Funktionen dar (vgl. 6.2).

Aufgrund der fehlenden Skalierbarkeit wurde auf Koordinaten und statische Bildwiederholraten verzichtet. Stattdessen wurde auf dynamisch skalierte Vektoren und Prozentwerte gesetzt. Zu den wichtigsten Interaktionsmustern und Heuristiken zählen What You See Is What You Get, Direct Manipulation und bekannte Prinzipien aus dem UI-Design und Web-Kontext, wie beispielsweise die zehn Heuristiken von Jakob Nielsen (vgl. 5.2). Zur Datenverwaltung und Weitergabe wurden ebenfalls Standards implementiert. Dateien werden im .glb-Format übergeben und Konfigurationsdateien werden in JSON-Dateien verwaltet, sodass sie auch manuell angepasst werden könnten.

8.2 Zusammenfassung

Zusammenfassend ist festzustellen, dass ein visueller Editor als zentraler Hebel für die Usability in der Konfiguration fungieren kann. Dadurch, dass Nutzer die Komponenten des 3D-Modells visuell identifizieren, auswählen und intuitiv verschieben können, kann die Konfiguration schneller, effizienter und weniger frustrierend vorgenommen werden (vgl. Test). Die Arbeit hat ergeben, dass ein Zusammenspiel vieler Heuristiken und Komponenten wichtig ist, um eine erfolgreiche Implementation vorzunehmen. Zudem konnte die bestehende Arbeitswei-

se validiert werden, da Erweiterungen des bestehenden Pakets ohne großflächige Umstellungen der Architektur übernommen werden konnten. Zudem konnte das Paket erfolgreich in eine Webseite implementiert und veröffentlicht werden.

8.3 Ausblick

In weiteren Iterationen könnten die bestehenden Limitationen mitigiert werden (vgl. 7.4). Beispielsweise könnten umfangreiche Nutzertests und Analyse-Tools auf der Webseite implementiert werden, um die Effekte der Verbesserungen und möglicherweise weitere Probleme zu identifizieren und zu bewerten. Zudem könnte der Funktionsumfang erweitert werden. Umfangreichere Funktionen zu Shadern und Belichtung könnten das Projekt weiter verbessern und hochertigere Darstellungen oder Stilanpassungen zu ermöglichen. Ebenfalls könnte ein Algorithmus konzipiert werden, welcher automatische Konfigurationen und Presets ermöglicht, dadurch könnte die Nutzung und Konfiguration des Frameworks weiter erleichtert werden.

Literatur

- ABBY Europe GmbH. (2022). 94% of companies lose potential customers during the digital onboarding process. Verfügbar 21. Januar 2026 unter <https://www.abby.com/de/company/news/94-perc-of-companies-lose-potential-customers-during-the-digital-onboarding-process/>
- Garnier, J. (2025). *Animejs documentation*. Verfügbar 17. Februar 2026 unter <https://animejs.com/easing-editor/spring/default>
- GitHub Docs. (n.d.). *Mit der npm-Registry arbeiten*. GitHub, Inc. Verfügbar 1. Februar 2026 unter <https://docs.github.com/de/packages/working-with-a-github-packages-registry/working-with-the-npm-registry>
- International Organization for Standardization. (n.d.). *Technical drawings – Projection methods – Part 2: Orthographic representations*.
- International Organization for Standardization. (2018). *Ergonomics of human-system interaction – Part 11: Usability: Definitions and concepts*.
- Jiang, Z., Chan, J., Tan, B. C. Y., & Chua, W. S. (2010). Effects of Interactivity on Website Involvement and Purchase Intention. *Journal of the Association for Information Systems*, 11(1). <https://doi.org/10.17705/1jais.00218>
- Kellner, M. E. (2025a). *default-options.js*. Verfügbar 1. Februar 2026 unter <https://github.com/MaximilianKellner/exploded-view-web-package/blob/main/Prototyp/src/config/default-options.js>
- Kellner, M. E. (2025b). *Exploded View Web Package Repository* (Version 2.0.1). GitHub. Verfügbar 3. Februar 2026 unter <https://github.com/MaximilianKellner/exploded-view-web-package>
- Kellner, M. E. (2025c). *Namenskonvention*. Verfügbar 20. Januar 2026 unter <https://github.com/MaximilianKellner/exploded-view-web-package/wiki/Namenskonvention>
- Kellner, M. E. (2025d, Oktober). *Konzeption und Entwicklung eines interaktiven (Visualisierungs-)Systems für Exploded Views im Web*, Technische Hochschule Köln. <https://github.com/MaximilianKellner/exploded-view-web-package/blob/main/Konzeption-und-Entwicklung-eines-interaktiven-Visualisierungs-Systems-f%C3%BCr-Exploded-Views-im-Web-Maximilian-Kellner.pdf>
- Kellner, M. E. (2026a). *Personas*. Verfügbar 26. Januar 2026 unter <https://github.com/MaximilianKellner/exploded-view-web-editor/wiki/Personas>
- Kellner, M. E. (2026b). *POC-Format-Beispiel*. Verfügbar 26. Januar 2026 unter <https://github.com/MaximilianKellner/exploded-view-web-editor/blob/main/POC/zeitachse/poc-Zeitachse.md>
- Kendrick, A. (2018, 21. Oktober). *Microinteractions in User Experience*. Nielsen Norman Group. Verfügbar 8. Februar 2026 unter <https://www.nngroup.com/articles/microinteractions/>
- Krug, S. (2014). *Don't Make Me Think, Revisited: A Common Sense Approach to Web Usability*. New Riders. Verfügbar 21. Januar 2026 unter <https://archive.org/details/SteveKrugDontMakeMeThink/page/n51/mode/2up>
- Loser, S. (2015). *Shaderbasierende Beleuchtung in der Echtzeitgrafik* [Bachelorarbeit]. Hochschule Mittweida, University of Applied Sciences [Studiengang Medieninformatik und Interaktives Entertainment]. Verfügbar 20.

- Januar 2026 unter https://monami.hs-mittweida.de/frontdoor/deliver/index/docId/5799/file/Bachelorarbeit_Shaderbasierende_Beleuchtung_in_Echtzeitumgebungen_Stefan_Loser.pdf
- Myers, B. A. (1998). A brief history of human-computer interaction technology. *Interactions*, 5(2). <https://doi.org/10.1145/274430.274436>
- Nielsen, J. (2006). *Progressive Disclosure*. Nielsen Norman Group. Verfügbar 18. Februar 2026 unter <https://www.nngroup.com/articles/progressive-disclosure/>
- Nielsen, J. (2024, Januar). 10 Usability Heuristics for User Interface Design. Verfügbar 21. Januar 2026 unter <https://www.nngroup.com/articles/ten-usability-heuristics/>
- Ozturk, B., & Akyuz, A. O. (2010). Realistic Lighting for Interactive Applications Using Semi-Dynamic Light Maps. *The Computer Games Journal*. Verfügbar 20. Januar 2026 unter https://user.ceng.metu.edu.tr/~akyuz/files/CG_2020_Lightmaps.pdf
- Rubinstein, J. S., Meyer, D. E., & Evans, J. E. (2001). Executive control of cognitive processes in task switching. *Journal of Experimental Psychology: Human Perception and Performance*, 27(4), 763–797. <https://doi.org/10.1037/0096-1523.27.4.763>
- Shneiderman, B. (1983). Direct Manipulation: A Step Beyond Programming Languages. *Computer*, 16(8). Verfügbar 21. Januar 2026 unter <https://www.cs.umd.edu/~ben/papers/Shneiderman1983Direct.pdf>
- Summerlin, R., & Powell, W. (2022). Effect of Interactivity Level and Price on Online Purchase Intention. *Journal of Theoretical and Applied Electronic Commerce Research*, 17(2), 652–668. <https://doi.org/10.3390/jtaer17020034>
- Three.js. (n.d. a). *DirectionalLightHelper: Three.js Documentation*. Three.js. Verfügbar 15. Februar 2026 unter <https://threejs.org/docs/#api/en/helpers/DirectionalLightHelper>
- Three.js. (n.d. b). *TransformControls: Three.js Documentation*. Three.js. Verfügbar 14. Februar 2026 unter <https://threejs.org/docs/#examples/en/controls/TransformControls>